

第 38 章: Managed Attributes (管理属性)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 这是全书高级可选部分的核心章之一。它把前文零散出现的属性拦截技术 (property、描述符、get attr/getattribute) 收拢成一张完整地图, 并以 CardHolder 校验案例做四路对照。学完本章, 你将真正理解 slots 与 property 的底层、能读懂 ORM 字段与框架中的“属性魔法”, 并为第 39 章装饰器、第 40 章元类打下地基。

38.1 开篇引言

英文原文

This chapter expands on the attribute interception techniques introduced earlier, introduces another, and employs them in a handful of larger examples. Like everything in this part of the book, this chapter is classified as an advanced topic and optional reading, because most applications programmers don't need to care about the material discussed here—they can fetch and set attributes on objects without concern for attribute implementations. Especially for tools builders, though, managing attribute access can be an importa

nt part of flexible APIs. Moreover, an understanding of the descriptor model covered here can make related tools such as slots and properties more tangible and may even be required reading if it appears in code you must use.

中文翻译

本章扩展了前文已介绍的属性拦截 (attribute interception) 技术，再引入另一种，并把它们用在若干更大的示例中。与本书这一部分的其他内容一样，本章属于进阶主题、可选阅读——多数应用程序员不必深入：他们可以读写对象属性，而不关心属性如何实现。但对工具构建者 (tools builders) 而言，管理属性访问往往是灵活 API 的关键。理解本章的描述符 (descriptor) 模型，也能让 slots、property 等工具更可感；若你维护的代码用到了它们，本章甚至可能是必读。

深度理解

- 核心概念：管理属性 (managed attributes) 让你在 `get / set / delete` 时自动运行逻辑，客户端仍写 `obj.attr`，不必改成 `getattr()`。
- 底层实现：四条技术钩在不同位置——`property`/描述符管具名属性；`getattr` 管找不到的读取；`getattribute` 管所有读取；`setattr` 管赋值。
- 设计原因：接口先保持简单字段，日后可无感升级为校验/计算/日志，而不破坏调用方（接口演化）。
- 实际问题：ORM 字段、配置校验、懒加载、代理委托、

只读视图。

·初学者误区：以为属性 = 实例 dict 的键；忽略描述符优先级后会出现“赋了值却读到别的东西”。

38.2 Why Manage Attributes? (为什么要管理属性?)

英文原文

Object attributes are central to most Python programs—they are where we often store information about the entities our scripts process. Normally, attributes are simply names for objects; a person's name attribute, for example, might be a simple string, fetched and set with basic attribute syntax:

```
python person.name # Fetch attribute value
person.name = value # Change attribute value
```

In most cases, the attribute lives in the object itself or is inherited from a class from which it derives. That basic model suffices for most programs you will write in your Python career. Sometimes, though, more flexibility is required. Suppose you've written a program to use a name attribute directly, but then your requirements change—for example, you decide that names must be validated or mutated with program logic when accessed. It's straightforward to code methods to manage access to the attribute's value (valid and tran

sform are abstract and hypothetical here):

```
python class Person: def getName(self): if not valid():  
raise TypeError('cannot fetch name') else: return self.  
name.transform() def setName(self, value): if not vali  
d(value): raise TypeError('cannot change name') else:  
self.name = transform(value) person = Person() perso  
n.getName() person.setName('value')
```

The problem with this is that it also requires changing all the places where names are used in the entire program—a possibly nontrivial task. Moreover, this approach requires the program to be aware of how values are exported: as simple names or called methods. If you begin with a method-based interface to data, clients are immune to changes; if you do not, changes can become problematic. This issue can crop up more often than you might expect. The value of a cell in a spreadsheet-like program, for instance, might begin its life as a simple discrete value but later mutate into an arbitrary calculation. Since an object's interface should be flexible enough to support such future changes without breaking existing code, switching to methods later is less than ideal.

中文翻译

对象属性是多数 Python 程序的核心——实体信息常常存放在属性里。通常属性只是对象的名字：例如一个人的 name 可能只是字符串，用基础语法读写即可。多数情况下，属性位于对象自身或由其类继承而来；这一模型对你职业生

涯中的大多数程序已足够。但有时需要更多灵活性。假设程序一开始直接使用 `name` 属性，后来需求变了——例如访问时必须校验或变换。当然可以写成 `getName / setName` 方法（这里的 `valid` 与 `transform` 是抽象假设），但这要求改遍整个程序中所有使用该名字的地方——可能是一项不小的工程。而且客户端必须知道数据是以“简单名字”还是“调用方法”导出的。若一开始就用方法接口，客户端可免疫变化；否则事后切换代价很高。这个问题比你想象的更常见。电子表单元格的值可能最初是简单离散值，后来变成任意计算。对象的接口应足够灵活，以支持未来变化而不破坏既有代码——因此“以后再改成方法”并不理想。

深度理解

- 核心概念：属性管理解决的是接口稳定性：对外仍是 `obj.name`，对内可随时插入校验、变换、计算。
- 底层实现：普通属性查找走实例 `dict` → 类 → 基类；管理属性则在查找链上插入钩子（`descriptor` 协议或运算符重载）。
- 设计原因：避免“从字段改方法”时的大规模重构；符合“开放-封闭”思想——对扩展开放、对调用方封闭。
- 实际问题：API 演进、数据校验、懒计算、访问审计。
- 初学者误区：一上来就全用 `getter/setter`（Java 风格）。Python 更推崇先用简单属性，需要时再升级为 `property`/描述符。

代码分析

```
class Person:
    def getName(self):
        if not valid():
            raise TypeError('cannot fetch name')
        else:
            return self.name.transform()
    def setName(self, value):
        if not valid(value):
            raise TypeError('cannot change name')
        else:
            self.name = transform(value)

person = Person()
person.getName()
person.setName('value')
```

·解释器对 `person.getName()` 走正常方法调用；对 `person.name` 则走属性查找。若一开始用了 `person.name`，事后改成方法，所有调用点都要改。

·设计思想：把“访问时逻辑”藏在属性协议里，调用方语法不变。

38.3 Inserting Code to Run on Attribute Access（在属性访问时插入代码）

英文原文

A better solution would allow you to run code automatically on attribute access if needed. That's one of the main roles of managed attributes—they provide wa

ys to add attribute accessor logic after the fact. More generally, they support arbitrary attribute usage modes that go beyond simple data storage. At various points in this book, we've met Python tools that allow our scripts to dynamically compute attribute values when fetching them and validate or change attribute values when storing them. In this chapter, we're going to focus more deeply on the tools already introduced, explore other tools in this category, and study some larger use-case examples in this domain. Specifically, this chapter presents four accessor techniques: 1. The property built-in, for specifying methods to handle access to a specific attribute 2. The get and set descriptor methods, for handling access to a specific attribute and the basis for other tools such as properties and slots 3. The getattr and setattr methods, for handling undefined attribute fetches and all attribute assignments 4. The getattribute method, for handling all attribute fetches We met these tools in Chapters 30 and 32 briefly, and in some cases, hardly at all. As you'll see here, all four techniques share goals to some degree, and it's usually possible to code a given problem using any one of them. That said, they also differ in some important ways. For example, the last two techniques listed here apply to specific attributes, whereas the first two are generic enough to be used by delegation-based proxy classes that must route arbitrary attributes to wrapped objects. As you'll find, all four sc

hemes also differ in both complexity and aesthetics in ways you must see in action to judge for yourself. Besides studying the specifics behind these four attribute interception techniques, this chapter also presents an opportunity to explore programs larger than most we've seen elsewhere in this book. The CardHolder case study at the end, for example, should serve as a self-study example of larger classes in action. We'll also be using some of the techniques outlined here in the next chapter to code decorators, so be sure you have at least a general understanding of these topics before you move on.

中文翻译

更好的方案是：在需要时，让代码在属性访问时自动运行。这正是管理属性的主要角色之一——事后为属性添加访问器逻辑。更一般地，它们支持超越简单数据存储的任意属性用法。本书多处已介绍过：取属性时可动态计算值，存属性时可校验或变换。本章将更深入已介绍的工具，探索同类其他工具，并研究更大的用例。具体而言，本章给出四种访问器技术：1. `property` 内建：为特定属性指定处理访问的方法2. 描述符的 `get / set`：处理特定属性访问，也是 `property`、`slots` 等工具的基础3. `getattr / setattr`：处理未定义属性读取，以及所有属性赋值4. `getattr`：处理所有属性读取这些工具在第 30、32 章曾简要（有时几乎未）提及。你会看到：四者目标有重叠，同一问题通常可用任一种实现；但差异也很重要——例如 `property`/描述符针对特定属性，而 `getattr / getattr` 足够通用，可用于必须把任

意属性路由到被包装对象的委托代理类。四者在复杂度与美学上也不同，需实际代码才能自行判断。除技术细节外，本章还提供比本书多数示例更大的程序。末尾的 CardHolder 案例可作自学材料。下一章编码装饰器也会用到这里的技术，请至少建立总体理解后再继续。

深度理解

- 核心概念：四种钩子 = 一张“属性访问路由图”。选型取决于：管一个名字还是所有名字？要不要管赋值？要不要出现在 dir 里？
- 底层实现：property/描述符实现描述符协议；getattr 等是运算符重载，走另一条路径。
- 设计原因：Python 提供多条路径，是为了在“精确控制单个字段”与“通用代理/委托”之间都能优雅表达。
- 实际问题：工具作者做校验框架、ORM、代理；应用作者读懂框架源码。
- 初学者误区：原文有一处笔误倾向——“后两种管特定、前两种管通用”与后文实际相反。正确记忆：property/描述符 = 特定属性；getattr/getattribute = 通用拦截。

38.4 Properties (property)

英文原文

Up first, the property protocol allows us to route a specific attribute's get, set, and delete operations to functions or methods we provide, enabling us to insert

code to be run automatically on attribute accesses, in intercept attribute deletions, and provide documentation for attributes if desired. As introduced in Chapter 32, properties are created with the property built-in and are assigned to class attributes, just like method functions. Accordingly, they are inherited by subclasses and instances, like any other class attributes. Their access-interception functions are provided with the self instance argument, which grants access to state information and class attributes available on the subject instance. A property manages a single, specific attribute; although it can't catch all attribute accesses generically, it allows us to control both fetch and assignment accesses and enables us to change an attribute from simple data to a computation freely without breaking existing code. As you'll see, properties are strongly related to descriptors; in fact, they are essentially a restricted form of them.

中文翻译

首先，property 协议允许我们把特定属性的 get、set、delete 操作路由到我们提供的函数或方法，从而在访问时自动运行代码、拦截删除，并按需提供属性文档。如第 32 章所述，property 由 property 内建创建，并像方法一样赋给类属性。因此它们被子类与实例继承。访问拦截函数会收到 self 实例参数，从而可访问该实例上的状态与类属性。一个 property 管理单个、特定属性：虽不能泛化捕获所有访问，但能同时控制读取与赋值，并允许把简单数据自由改成计

算而不破坏既有代码。property 与描述符关系密切——本质上是描述符的一种受限形式。

深度理解

- 核心概念：property = “给某一个名字挂上 get/set/delete 三个钩子” 的便捷 API。
- 底层实现：property 对象本身是数据描述符（实现了 get/set/delete）。
- 设计原因：让 “字段升级为计算/校验” 时，调用方语法仍是 obj.x。
- 实际问题：只读属性、带校验的写入、计算属性（如 are a）。
- 初学者误区：把 property 当 “Java 的 getter/setter 语法糖” 而滥用；Python 惯例是需要时才用。

38.5 The Basics (Properties 基础)

英文原文

A property is created by assigning the result of a built-in function to a class attribute: `python attribute = property(fget, fset, fdel, doc)` None of this built-in's arguments are required, and all default to None if not passed. For the first three, this None means that the corresponding operation is not supported, and attempting it will raise an `AttributeError` exception automatically. When these arguments are used, we pass `fget`

a function for intercepting attribute fetches, fset a function for assignments, and fdel a function for attribute deletions. Technically, all three of these arguments accept any callable, including a class's method, having a first argument to receive the instance being qualified. When later invoked, the fget function returns the computed attribute value, fset and fdel return nothing (really, None), and all three may raise exceptions to reject access requests. The doc argument receives a documentation string for the attribute if desired. If omitted, the property copies the docstring of the fget function, which, as usual, defaults to None. This built-in property call returns a property object, which we assign to the name of the attribute to be managed in the class scope, where it will be inherited by every instance. As you'll learn ahead, this assignment can be automated by @ decorator syntax, though its distributed usage may seem awkward for set and delete methods. However assigned, later accesses to the attribute automatically invoke the property's handlers.

中文翻译

创建 property 的方式是：把内建函数的返回值赋给类属性：
`python attribute = property(fget, fset, fdel, doc)` 四个参数均可省略，默认 None。对前三个而言，None 表示不支持该操作，尝试时自动抛出 `AttributeError`。使用时：
fget 拦截读取，fset 拦截赋值，fdel 拦截删除。技术上，三者都接受任意可调用对象（含类方法），第一个参数接收

被限定的实例。调用时：fget 返回计算后的属性值；fset/fdel 返回 None；三者都可抛异常拒绝访问。doc 为属性文档字符串；若省略，则复制 fget 的 docstring（通常默认 None）。property(...) 返回 property 对象，在类作用域中赋给要管理的属性名，由每个实例继承。后文会看到可用 @ 装饰器语法自动完成该赋值，但对 set/delete 方法来说，分散写法有时显得别扭。无论如何赋值，之后对该属性的访问都会自动调用 property 的处理函数。

深度理解

- 核心概念：property(fget, fset, fdel, doc) 把四个“槽位”填进一个描述符对象。
- 底层实现：实例上 obj.attr → 找到类上的 property → 调 fget(obj)；obj.attr = v → fset(obj, v)。
- 设计原因：缺省 None 即“只读/不可删”，符合最小惊讶。
- 实际问题：只需 getter 时写 property(getX)；需要文档时传 doc 或依赖 getter 的 docstring。
- 初学者误区：把数据存在与 property 同名的实例属性上，导致递归或遮蔽——通常用 name 存真实数据。

代码分析

```
attribute = property(fget, fset, fdel, doc)
```

- 解释器在类体执行时创建 property 对象并绑定到类字典。
- 之后实例访问不查实例 dict 里的同名键优先于数据描述

符——若实现了 set，赋值也会进 property，而不是写进实例字典。

38.6 A First Example (Properties 第一个示例)

英文原文

To demonstrate how this translates to working code, the class in Example 38-1 uses a property to trace access to an attribute named name; the actual stored data is named name so it does not clash with the property.

Example 38-1. prop-person.py

```
python class Person: def init(self, name): self.name =  
name def getName(self): print('fetch...') return self.name  
def setName(self, value): print('change...') self.name =  
value def delName(self): print('remove...') del self.name
```

```
name = property(getName, setName, delName, 'name property docs')  
sue = Person('Sue Jones') # sue has a managed attribute  
print(sue.name) # Runs getName  
sue.name = 'Susan Jones' # Runs setName  
print(sue.name) del sue.name # Runs delName
```

```
print('-' * 20)  
bob = Person('Bob Smith') # bob inherits property too  
print(bob.name) print(Person.name.doc)  
# Or help(Person.name) This particular property does
```

n't do much—it simply intercepts and traces an attribute—but it serves to demonstrate the protocol. When this code is run, two instances inherit the property, just as they would any other attribute attached to their class. However, accesses to their name attribute are caught and managed by the code we provide:

```
$ python3 prop-person.py fetch... Sue Jones change..  
. fetch... Susan Jones remove... -----fetch..  
.. Bob Smith name property docs
```

Like all class attributes, properties are inherited by both instances and lower subclasses. If we change our example as follows, for instance:

```
python class Super: # the original Person class code ..  
. name = property(getName, setName, delName, 'na  
me property docs') class Person(Super): pass # Prope  
rties are inherited (class attrs)
```

sue = Person('Sue Jones') # rest unchanged the output is the same—the Person subclass inherits the name property from Super, and the sue instance gets it from Person. In terms of inheritance, properties work the same as normal methods; because they have access to the self instance argument, they can access instance state information and methods irrespective of subclass depth, as the next section further demonstrates.

中文翻译

为演示如何写成可运行代码，示例 38-1 用 property 追踪对 name 属性的访问；真实数据存在 name，以免与 property 冲突。示例 38-1. prop-person.py（见英文原文代码块）这个 property 本身做的事不多——只是拦截并打印追踪——但足以演示协议。运行时，两个实例都继承该 property（与任何类属性一样），但对 name 的访问被我们提供的代码捕获并管理。输出如英文原文所示。与所有类属性一样，property 被实例与更低层子类继承。若把 name = property(...) 放到 Super，Person(Super) 只写 pass，输出不变——子类继承 property，实例再从子类获得。就继承而言，property 与普通方法相同；因持有 self，不论子类深度，都能访问实例状态与方法。

深度理解

- 核心概念：name 存数据，name 是管理入口——双名模式是 property 的常规写法。
- 底层实现：sue.name → Person.dict['name'].get(sue, Person) → getName(sue)。
- 设计原因：避免 self.name = ... 在 getter/setter 内部再次触发 property 造成递归。
- 实际问题：日志、审计、惰性迁移旧字段。
- 初学者误区：在 init 里写 self.name = name 会走 setter（有时是故意的，用于统一校验；有时应用 name 直接写）。

代码分析

```
class Person:
    def __init__(self, name):
        self._name = name
    def getName(self):
        print('fetch...')
        return self._name
    def setName(self, value):
        print('change...')
        self._name = value
    def delName(self):
        print('remove...')
        del self._name
    name = property(getName, setName, delName, 'name property docs...')
```

- bob 与 sue 共享同一个 property 对象（类属性），但各自有独立的 name。
- Person.name.doc 读取 property 的文档字符串。

38.7 Computed Attributes (Properties 计算属性)

英文原文

The example in the prior section simply traces attribute accesses. Usually, though, properties do much more—computing the value of an attribute dynamically when fetched, for instance, as Example 38-2 illustrates. Example 38-2. prop-computed.py

```
python class PropSquare: def init(self, start): self.valu
```

```
e = start def getX(self): return self.value 2 # On attr fetch
def setX(self, value): self.value = value # On attr assign
X = property(getX, setX) # No delete or docs
P = PropSquare(3) # Two instances of class with property
Q = PropSquare(32) # Each has different state information
print(P.X) # 3 2 P.X = 4 print(P.X) # 4 2 print(Q.X) # 32 2 (1024)
This class defines an attribute X that is accessed as though it were simple data, but really runs code to compute its value when fetched. The net effect triggers an implicit method call. When the code is run, the value is stored in the instance as state information, but each time we fetch it via the managed attribute, its value is automatically squared:
```

```
$ python3 prop-computed.py 9 16 1024
```

Notice that we've made two different instances—because property methods automatically receive a self argument, they have access to the state information stored in instances. In our case, this means the fetch computes the square of the subject instance's own data

中文翻译

上一节的示例只是追踪访问。通常 property 能做更多——例如在读取时动态计算属性值，如示例 38-2。示例 38-2. prop-computed.py（见英文原文）该类定义属性 X：访问时像简单数据，实际在读取时运行代码计算。净效果是触发隐式方法调用。运行时，值作为状态存在实例上，但每次通过管理属性读取时都会自动平方。输出：9、16、1024。

注意我们创建了两个不同实例——property 方法自动收到 self，因此能访问各自实例状态；本例中读取会计算当前实例自己的数据的平方。

深度理解

- 核心概念：计算属性（computed attribute）——对外是字段，对内是函数。
- 底层实现：P.X 不读 dict['X']，而调 getX(P) 返回 P.value 2。
- 设计原因：隐藏“平方”细节，调用方仍写 P.X。
- 实际问题：area、fullname、缓存失效后的重算。
- 初学者误区：在 getter 里做极重计算却无缓存，导致每次访问都很慢——需要时用 functools.cachedproperty（现代实践，书中未展开）。

代码分析

```
class PropSquare:
    def __init__(self, start):
        self.value = start
    def getX(self):
        return self.value ** 2
    def setX(self, value):
        self.value = value
    X = property(getX, setX)

P = PropSquare(3)
Q = PropSquare(32)
print(P.X)    # 9
P.X = 4
print(P.X)    # 16
```

- P 与 Q 状态独立：self 保证计算绑在正确实例上。

38.8 Coding Properties with Decorators (用装饰器编写 property)

英文原文

Although we're saving additional details until the next chapter, we introduced function decorator basics earlier, in Chapter 32. Recall that the function decorator syntax: `python @decorator def func(args): ...` is automatically translated to this equivalent by Python to rebind the function name to the result of the decorator callable: `python def func(args): ...func = decorator(func)` Because of this mapping, the `property` built-in can automatically serve as a decorator to define a function that will run automatically when an attribute is fetched: `python class Person: @property def name(self): ...` # Rebinds: `name = property(name)` When run, the decorated method is automatically passed to the first argument of the `property` built-in. This is really just alternative syntax for creating a property and rebinding the attribute name manually, but may be seen as more explicit in this role: `python class Person: def name(self): ...name = property(name)` # Manual equivalent to `@property`

中文翻译

更多细节留到下一章，但第 32 章已介绍函数装饰器基础。回顾：@decorator 写在 def func 前，等价于定义后执行 func = decorator(func)。因这一映射，property 内建可直接作装饰器，定义“读取属性时自动运行”的函数：python class Person: @property def name(self): ... # 重绑定: name = property(name) 运行时，被装饰方法自动传入 property 的第一个参数。这只是手动创建 property 并重绑定名字的另一种语法，但在此角色中可能更显式。

深度理解

- 核心概念：@property = name = property(name) 的语法糖（名字重绑定）。
- 底层实现：类体执行到 @property def name 时，先创建函数，再 name = property(name)。
- 设计原因：把“这是个只读/受管属性”写在定义处，提高可读性。
- 实际问题：现代 Python 代码中最常见的 property 写法。
- 初学者误区：以为 @property 与手动 property(...) 机制不同——其实完全等价。

38.9 Setter and deleter decorators (setter 与 deleter 装饰器)

英文原文

The preceding works naturally for property get functions, but what about other accesses? In full detail, property objects also have getter, setter, and deleter methods that assign the corresponding property access or methods and return a copy of the property itself. We can use these to specify components of properties by decorating normal methods, too, though the getter component (along with attributes docs) is usually filled in automatically by the act of creating the property itself.

Example 38-3. prop-person-deco.py

```
python class Person:
    def init(self, name):
        self.name = name
    @property
    def name(self):
        # name = property(
            name)
        name property docs'

    print('fetch...')
    return self.name
    @name.setter
    def name(self, value):
        # name = name.setter(name)
        print('change...')
        self.name = value
    @name.deleter
    def name(self):
        # name = name.deleter(name)
        print('remove...')
        del self.name

sue = Person('Sue Jones')
print(sue.name) # Runs name getter
sue.name = 'Susan Jones' # Runs name setter
print(sue.name)
del sue.name # Runs name deleter

print('-' * 20)
bob = Person('Bob Smith')
print(bob.name)
print(Person.name.doc) # Or help(Person.name)
In fact, this code is equivalent to the first example in thi
```

s section—decoration is just an alternative way to code properties in this case. When it's run, the results are the same:

```
$ python3 prop-person-deco.py fetch... Sue Jones change... fetch... Susan Jones remove... -----  
-fetch... Bob Smith name property docs
```

Compared to manual assignment of property results, using decorators to properties in this example requires just three extra lines of code—a seemingly negligible difference. As is so often the case with alternative tools, though, the choice between the two techniques is largely subjective.

中文翻译

前述写法自然适合 property 的 get 函数，那其他访问呢？完整地说，property 对象还有 getter、setter、deleter 方法：它们设置对应访问器并返回 property 自身的副本。我们也可通过装饰普通方法来指定 property 的各组件；不过 getter（以及文档）通常在创建 property 时已自动填好。示例 38-3. prop-person-deco.py（见英文原文）与本节第一个示例等价——装饰只是另一种写法。运行结果相同。与手动赋值相比，本例用装饰器只多约三行——差异可忽略。与许多替代工具一样，二者选择 largely 主观。

深度理解

·核心概念：@name.setter / @name.deleter 在已有 property 上追加组件，并重新绑定同名。

- 底层实现：name.setter(func) 返回新 property（或同一增强对象），再赋回 name。
- 设计原因：三个同名 def name 在类体中依次重绑定，最终留下带齐 get/set/del 的 property。
- 实际问题：可读的“属性三段式”定义。
- 初学者误区：先写 @name.setter 再写 @property 会失败——必须先有 property 对象。

代码分析

```
@property
def name(self): ...

@name.setter
def name(self, value): ...

@name.deleter
def name(self): ...
```

- 类体执行顺序：① name = property(fget) ② name = name.setter(fset) ③ name = name.deleter(fdel)。
- 三个 def 同名不冲突，因为每次都被重绑定到 property，而不是留下三个函数。

38.10 Descriptors（描述符）

英文原文

Very briefly previewed in Chapter 32, descriptors provide an alternative way to intercept attribute access; they are strongly related to the properties discussed in the prior section. Really, a property is a kind of des

criptor—technically speaking, the property built-in is just a simplified way to create a specific type of descriptor that runs method functions on attribute accesses. In fact, descriptors are the underlying implementation mechanism for a variety of class tools, including both properties and slots, and play other internal roles in Python that we can safely skip here. Functionally speaking, the descriptor protocol allows us to route a specific attribute's get, set, and delete operations to methods of a separate class's instance object that we provide. This allows us to insert code to be run automatically on attribute fetches and assignments, intercept attribute deletions, and provide documentation for the attributes if desired. Descriptors are created as independent classes, and they are assigned to class attributes just like method functions. Like any other class attribute, they are inherited by subclasses and instances. Their access-interception methods are provided with both a self for the descriptor instance itself as well as the instance of the client class whose attribute references the descriptor object. Because of this, they can retain and use state information of their own, as well as state information of the subject instance. For example, a descriptor may call methods available in the client class, as well as descriptor-specific methods it defines. Like a property, a descriptor manages a single, specific attribute; although it can't catch all attribute accesses generically, it provides control over

r both fetch and assignment accesses and allows us to change an attribute name freely from simple data to a computation without breaking existing code. Properties can be coded as descriptors directly. Unlike properties, though, descriptors provide a more general tool. For instance, because they are coded as normal classes, descriptors have their own state, may participate in descriptor inheritance hierarchies, can use composition to aggregate objects, and provide a natural structure for coding internal methods and attribute documentation strings.

中文翻译

第 32 章曾极简预览过：描述符（descriptors）提供拦截属性访问的另一途径，与上一节的 property 关系密切。实际上，property 就是一种描述符——技术上，property 内建只是创建“在属性访问时运行方法”的特定描述符的简化方式。描述符是多种类工具（含 property 与 slots）的底层实现机制，并在 Python 内部扮演其他角色（此处可安全跳过）。功能上，描述符协议（descriptor protocol）把特定属性的 get/set/delete 路由到我们提供的另一个类的实例的方法上。从而可在读取/赋值时自动运行代码、拦截删除、提供文档。描述符写成独立类，像方法一样赋给类属性，由子类与实例继承。拦截方法既收到描述符自身的 self，也收到客户端类实例——因此可同时持有描述符状态与主体实例状态。例如可调用客户端类方法，也可调用描述符自有方法。与 property 一样，描述符管单个特定属性；虽不能泛化捕获一切，但能控制读写，并允许从简单数据改

为计算而不破坏代码。property 可直接用描述符实现。但描述符更通用：作为普通类，它们有自有状态、可参与继承层次、可用组合聚合对象，并为内部方法与文档字符串提供自然结构。

深度理解

- 核心概念：描述符 = “实现了 get / set / delete 的对象”，挂在类上时劫持该属性名的访问。
- 底层实现：type.getattribute 发现类字典中的描述符后，转调其 get 等。
- 设计原因：统一 property、slots、绑定方法、classmethod 等机制。
- 实际问题：可复用的字段类型（ORM Column、校验字段）。
- 初学者误区：把描述符实例赋给 self.xxx（实例属性）——无效；必须赋给类属性。

38.11 The Basics (Descriptors 基础)

英文原文

As mentioned, descriptors are coded as separate classes and provide specially named accessor methods for the attribute access operations they wish to intercept—get, set, and deletion methods in the descriptor class are automatically run when the attribute assigned to the descriptor class instance is accessed in the

corresponding way: python class Descriptor: "docstring goes here" def get(self, instance, owner): ... # Return attr value def set(self, instance, value): ... # Return nothing (None) def delete(self, instance): ... # Return nothing (None) Classes with any of these methods are considered descriptors, and their methods are special when one of their instances is assigned to another class's attribute—when the attribute is accessed, these methods are automatically invoked. If any of these methods are absent, it generally means that the corresponding type of access is not supported. Unlike properties, however, omitting a set allows the descriptor attribute's name to be assigned and thus redefined in an instance, thereby hiding the descriptor—to make an attribute read-only, you must define set to catch assignments and raise an exception. Descriptors with set methods also have some special-case implications for inheritance that we'll largely defer until Chapter 40's coverage of metaclasses and the complete inheritance specification. In short, a descriptor with a set is known formally as a data descriptor and is given precedence over other names located by normal inheritance rules. The inherited descriptor for attribute class, for example, overrides the same name in an instance's namespace dictionary. This also works to ensure that data descriptors you code in your own classes take precedence over others.

中文翻译

如前所述，描述符写成独立类，并用特殊命名的访问器方法拦截想处理的操作——当赋给描述符实例的属性被以相应方式访问时，描述符类中的 `get/set/delete` 方法自动运行（见英文原文协议骨架）。实现了其中任一方法的类即视为描述符；其实例赋给另一类的属性后，访问该属性会自动调用这些方法。某方法缺失通常表示不支持对应访问类型。但与 `property` 不同：省略 `set` 并不能让属性只读——仍可在实例上赋值从而遮蔽描述符。若要只读，必须定义 `set` 捕获赋值并抛异常。带 `set` 的描述符对继承还有特例（细节 `defer` 到第 40 章元类与完整继承规格）。简言之，带 `set` 的描述符正式称为数据描述符（`data descriptor`），优先于正常继承规则找到的其他名字。例如 `class` 的继承描述符会覆盖实例命名空间中的同名项。这也保证你自己写的描述符优先于其他名字。

深度理解

- 核心概念：数据描述符（有 `set` 或 `delete`）vs 非数据描述符（仅 `get`，如普通函数/方法）。
- 底层实现：查找优先级大致为：数据描述符 > 实例 `dict` > 非数据描述符 > `getattr`。
- 设计原因：保证 `property`、`slots` 等不被实例随意覆盖；方法（非数据）可被实例属性遮蔽。
- 实际问题：只读字段必须显式 `set` 抛错。
- 初学者误区：以为“没写 `set` = 只读”，结果 `obj.x = 1` 默默写进实例字典，下次读到的是 1 而不是描述符逻辑。

代码分析

```
class Descriptor:
    "docstring goes here"
    def __get__(self, instance, owner): ...
    def __set__(self, instance, value): ...
    def __delete__(self, instance): ...
```

三者都可缺；缺 set 时赋值走默认实例存储，隐藏类上描述符。

38.12 Descriptor method arguments (描述符方法参数)

英文原文

Before we code anything realistic, let's take a brief look at some fundamentals. All three descriptor methods outlined in the prior section are passed both the descriptor class instance (self) and the instance of the client class to which the descriptor instance is attached (instance). The get access method additionally receives an owner argument, specifying the class to which the descriptor instance is attached. Its instance argument is either the instance through which the attribute was accessed (for instance.attr), or None when the attribute is accessed through the owner class directly (for class.attr). The former of these generally computes a value for instance access, and the latter usually returns self if descriptor object access is supported

. For example, in the following REPL session, when `X.attr` is fetched, Python automatically runs the `get` method of the `Descriptor` class instance to which the `Subject.attr` class attribute is assigned:

```
python >>> class Descriptor: ... def get(self, instance, owner): ... print(self, instance, owner, sep='\n') ... >>> class Subject: ... attr = Descriptor() # Descriptor instance is class attr ... >>> X = Subject() >>> X.attr <main.Descriptor object at 0x...> <main.Subject object at 0x...> <class'main.Subject'> >>> Subject.attr <main.Descriptor object at 0x...> None <class'main.Subject'>
```

Notice the arguments automatically passed in to the `get` method in the first attribute fetch—when `X.attr` is fetched, it's as though the following translation occurs (though the `Subject.attr` here doesn't invoke `get` again as it normally would): `python X.attr => Descriptor.get(Subject.attr, X, Subject)` The descriptor knows it is being accessed directly when its instance argument is `None`.

中文翻译

在写真实代码前，先看基础。前述三个描述符方法都会收到：描述符类实例（`self`），以及描述符所附着的客户端类实例（`instance`）。`get` 额外收到 `owner`，即描述符所附着的类。`instance` 参数：通过实例访问（`instance.attr`）时为该实例；通过所有者类直接访问（`class.attr`）时为 `None`。前者通常为实例访问计算值；后者若支持“取描述符对象本身”，通常返回 `self`。REPL 示例（见英文原文）：`X.attr` 时自动运行 `Subject.attr` 所绑定的 `Descriptor` 实例的

get。参数打印显示：描述符、X、Subject；而 Subject.attr 时 instance 为 None。第一次读取时参数自动传入——X.attr 仿佛翻译为：python X.attr => Descriptor.get(Subject.attr, X, Subject) 当 instance 为 None 时，描述符知道自己是被直接（通过类）访问的。

深度理解

- 核心概念：get(self, instance, owner) 三参数定义了 “谁、从哪、属于哪类”。
- 底层实现：实例访问与类访问走同一 get，靠 instance is None 分支。
- 设计原因：支持 Class.attr 返回描述符自身（便于 help、检查），obj.attr 返回计算值。
- 实际问题：绑定方法就是函数描述符在 instance is not None 时返回绑定对象。
- 初学者误区：在 get 里忘记处理 instance is None，导致 Class.attr 崩溃。

代码分析

```
X.attr => Descriptor.__get__(Subject.attr, X, Subject)
```

- Subject.attr 在类字典中就是那个 Descriptor() 实例；通过实例取时不会先再触发一次无限 get（解释器特殊处理）。

38.13 Read-only descriptors (只读描述符)

英文原文

As mentioned earlier, unlike properties, simply omitting the set method in a descriptor isn't enough to make an attribute read-only because the descriptor name can be assigned in an instance. In the following, the attribute assignment to `X.a` stores `a` in the instance object `X`, thereby hiding the descriptor stored in class `C`:

```
python >> class D:
... def get(args): print('get') ... >> class C:
... a = D() # Attribute "a" is a descriptor instance ... >
> X = C() >> X.a # Runs inherited descriptor get
>> C.a get >> X.a = 99 # Stored on X, hiding C.a! >>
X.a 99 >> list(X.dict.keys()) ['a'] >> Y = C() >> Y.a # Y
still inherits descriptor get >> C.a get
```

This is the way all instance attribute assignments work in Python, and it allows classes to selectively override class-level defaults in their instances. To make a descriptor-based attribute read-only, catch the assignment in the descriptor class and raise an exception—when assigning an attribute that is a descriptor, Python effectively bypasses the normal instance-level assignment behavior and routes the operation to the descriptor object:

```
python >> class D:
```

```
... def get(args): print('get') ... def set(args): raise AttributeError('cannot set') ... >> class C:
```

```
... a = D() ... >> X = C() >> X.a # Routed to C.a.get  
get >> X.a = 99 # Routed to C.a.set  
AttributeError: cannot set
```

NOTE The deletion trio: Be careful not to confuse the descriptor delete method with the general del method. The former is called on attempts to delete the managed attribute name on an instance of the owner class; the latter is the general instance destructor method, run when an instance of any kind of class is about to be garbage-collected.

Descriptor delete is more closely related to the delattr generic attribute deletion method we'll study later in this chapter. See Chapter 30 for more on operator-overloading methods like del.

中文翻译

如前所述，与 property 不同，描述符里省略 set 不足以只读——仍可在实例上赋值。下例中，X.a = 99 把 a 存进实例 X，从而隐藏类 C 上的描述符（见英文原文 REPL）。这是 Python 实例属性赋值的常规行为，允许实例选择性覆盖类级默认值。要使基于描述符的属性只读，须在描述符类中捕获赋值并抛异常——当赋值目标是（数据）描述符时，Python 绕过常规实例级赋值，把操作路由到描述符对象（见带 set 抛 AttributeError 的示例）。注意删除三件套：勿

将描述符的 `delete` 与通用的 `del` 混淆。前者在删除所有者类实例上的受管属性名时调用；后者是实例析构方法，在任意类实例即将被垃圾回收时运行。`delete` 更接近后文的通用 `delattr`。`del` 详见第 30 章。

深度理解

- 核心概念：只读 = 必须实现 `set` 并拒绝；仅有 `get` = 可被实例属性覆盖的“默认计算”。
- 底层实现：有 `set` → 数据描述符 → 赋值不写 `dict`，进 `set`。
- 设计原因：非数据描述符允许实例覆盖（方法可被替换）；数据描述符保护字段语义。
- 实际问题：常量配置、只读视图。
- 初学者误区：`delete` vs `del` 名字相近，职责完全不同。

38.14 A First Example (Descriptors 第一个示例)

英文原文

To see how this all comes together in more realistic code, let's get started with the same first example we wrote for properties. Example 38-4 defines a descriptor that intercepts access to an attribute named `name` in its clients. Its methods use their instance argument to access state information in the subject instance, where the name string is actually stored. Example 38-4.

desc-person.py

```
python class Name: 'name descriptor docs'
```

```
def get(self, instance, owner): print('fetch...') return in  
stance.name def set(self, instance, value): print('chan  
ge...') instance.name = value
```

```
def delete(self, instance): print('remove...') del instanc  
e.name
```

```
class Person: def init(self, name): self.name = name n  
ame = Name() # Assign descriptor to attr sue = Perso  
n('Sue Jones') print(sue.name) # Runs Name.get sue.  
name = 'Susan Jones' # Runs Name.set print(sue.nam  
e) del sue.name # Runs Name.delete
```

```
print('-'20) bob = Person('Bob Smith') print(bob.nam  
e) print(Name.doc) # Or help(Name) Notice in this co  
de how we assign an instance of our descriptor class  
to a class attribute in the client class; because of this,  
it is inherited by all instances of the class, just like a c  
lass's methods. Really, we must assign the descriptor  
to a class attribute like this—it won't work if assigned  
to a self instance attribute instead. When the descript  
or's get method is run, it is passed three objects to d  
efine its context: self is the Name class instance; insta  
nce is the Person class instance; owner is the Person  
class. When this code is run, the descriptor's method  
s intercept accesses to the attribute, much like the pr  
operty version. In fact, the output is the same again.  
Also like in the property example, our descriptor clas
```

s instance is a class attribute and thus is inherited by all instances of the client class and any subclasses. If we change the Person class to inherit from a Super that holds name = Name(), the output is the same. Also, note that when a descriptor class is not useful outside the client class, it's perfectly reasonable to embed the descriptor's definition inside its client syntactically (nested class). When coded this way, Name becomes a local variable in the scope of the Person class statement, such that it won't clash with any names outside the class. The testing code's last line must change to print(Person.Name.doc).

中文翻译

为看清更现实的代码如何拼在一起，我们从与 property 相同的第一个示例开始。示例 38-4 定义描述符，拦截客户端对 name 的访问；方法用 instance 参数访问主体实例中实际存储的名字字符串（完整代码见英文原文）。注意：我们把描述符类的实例赋给客户端类的类属性，从而被所有实例继承，与方法一样。必须这样赋给类属性——赋给 self 实例属性无效。get 运行时收到三对象：self = Name 实例；instance = Person 实例；owner = Person 类。运行输出与 property 版相同。描述符作为类属性也被子类继承。若描述符仅在客户端内有用，可嵌套定义在客户端类中，此时 Name 成为 Person 类语句作用域的局部名，不与外部冲突；测试最后一行改为 print(Person.Name.doc)。

深度理解

- 核心概念：name = Name() 在类体执行一次，所有 Person 实例共享同一个描述符对象。
- 底层实现：状态在 instance.name（每实例一份）；描述符对象本身无 per-instance 数据。
- 设计原因：共享描述符 + 实例状态 = 可复用字段类型。
- 实际问题：多个类共用同一个 Name 描述符类。
- 初学者误区：self.name = Name() 写在 init——不会触发描述符协议。

代码分析

```
class Name:
    def __get__(self, instance, owner):
        return instance._name
    def __set__(self, instance, value):
        instance._name = value
    def __delete__(self, instance):
        del instance._name

class Person:
    def __init__(self, name):
        self._name = name
    name = Name()    #
```

- sue.name → Name.get(namedescrptor, sue, Person)。
-

38.15 Computed Attributes (Descriptors 计算属性)

英文原文

As was the case when using properties, our first descriptor example of the prior section didn't do much—it simply printed trace messages for attribute accesses as a demo. In practice, descriptors can also be used to compute attribute values each time they are fetched. Example 38-5 illustrates—it's a rehash of the same example we coded for properties but uses a descriptor to automatically square an attribute's value each time it is fetched. Example 38-5. desc-computed.py

```
python class DescSquare: def init(self, start): # Each desc has own state self.value = start def get(self, instance, owner): # On attr fetch return self.value 2 def set(self, instance, value): # On attr assign self.value = value # No delete or docs class Client1: X = DescSquare(3) # Assign descriptor instance to class attr class Client2: X = DescSquare(32) # Another instance in another client class # Could also code two instances in same class c1 = Client1() c2 = Client2() print(c1.X) # 3 2 c1.X = 4 print(c1.X) # 4 2 print(c2.X) # 32 2 (1024) When run, the output of this example is the same as that of the original property-based version, but here a descriptor class object is intercepting the attribute accesses instead of a property:
```

```
$ python3 desc-computed.py 9 16 1024
```

中文翻译

与 `property` 时一样，上一节第一个描述符示例只是打印追踪。实践中描述符也可在每次读取时计算属性值。示例 38-5 是 `property` 版“平方”示例的重写，用描述符在每次读取时自动平方（代码见英文原文）。运行输出与 `property` 版相同（9、16、1024），但拦截者是描述符类对象而非 `property`。

深度理解

- 核心概念：状态可存在描述符自身（`self.value`），不依赖客户端实例。
- 底层实现：每个 `DescSquare(n)` 是独立对象；`Client1.X` 与 `Client2.X` 是两个描述符实例。
- 设计原因：演示“描述符状态”——与实例状态相对。
- 实际问题：跨实例共享的配置、计数器（慎用）。
- 初学者误区：把本应 `per-instance` 的员工姓名存在描述符上，导致所有人同名。

代码分析

```
class DescSquare:
    def __init__(self, start):
        self.value = start          #
    def __get__(self, instance, owner):
        return self.value ** 2
    def __set__(self, instance, value):
        self.value = value
```

·这里甚至未使用 instance——纯描述符状态计算。

38.16 Using State Information in Descriptors (在描述符中使用状态信息)

英文原文

If you closely study the two descriptor examples we've written so far, you might notice that they get their information from different places—the first (the name attribute example) uses data stored on the client instance, and the second (the attribute squaring example) uses data attached to the descriptor object itself (a.k.a. self). In fact, descriptors can use both instance state and descriptor state, or any combination thereof:

- Descriptor state is used to manage either data internal to the workings of the descriptor or data that spans all instances. It can vary per attribute appearance (often per client class).
- Instance state records information related to and possibly created by the client class. It can vary per client-class instance (that is, per application object). In other words, descriptor state is p

er-descriptor data, and instance state is per-client-instance data. As usual in OOP, you must choose state carefully. For example, you would not normally use descriptor state to record employee names since each client instance requires its own value—if stored in the descriptor, each client class instance will effectively share the same single copy. On the other hand, you would not usually use instance state to record data pertaining to descriptor implementation internals—if stored in each instance, there would be multiple varying copies. Descriptor methods may use either state form, but descriptor state sometimes makes it unnecessary to use special naming conventions to avoid name collisions in the instance for data that is not instance specific.

中文翻译

若仔细研究目前两个描述符示例，会发现信息来源不同：第一个（name）用客户端实例上的数据；第二个（平方）用描述符对象自身（即 self）上的数据。实际上，描述符可用实例状态、描述符状态，或任意组合：- 描述符状态（descriptor state）：管理描述符内部工作数据，或跨越所有实例的数据；可随属性出现点（常随客户端类）而变。- 实例状态（instance state）：记录与客户端类相关、可能由其创建的信息；可随客户端类实例（即应用对象）而变。换言之：描述符状态 = per-descriptor；实例状态 = per-client-instance。与 OOP 中一贯做法一样，须谨慎选择状态。例如通常不会用描述符状态记录员工姓名——每实例需要

自己的值，若存在描述符上则所有客户端实例共享同一份。反之，通常也不会用实例状态记录描述符实现内部数据——否则每实例多份可变副本。描述符方法可用任一形式；对非实例特定的数据，描述符状态有时使你不必用特殊命名约定来避免与实例名冲突。

深度理解

- 核心概念：状态放置是描述符设计的第一决策——共享 vs 每实例。
- 底层实现：描述符对象在类上唯一；`instance.xxx` 每对象一份。
- 设计原因：ORM 字段类型共享校验逻辑（描述符/类级），行数据存在实例。
- 实际问题：CardHolder 案例后文会演示“共享描述符状态写错”导致 bob 变成 sue。
- 初学者误区：图省事全放描述符上，多实例时数据串台。

38.17 描述符状态示例 (desc-state-desc.py)

英文原文

Example 38-6 attaches information to its own instance, so it doesn't clash with that on the client class's instance—but also shares that information between two client instances.

Example 38-6. desc-state-desc.py

```
python class DescState: # Use descriptor state def init(self, value): self.value = value def get(self, instance, owner): # On attr fetch print('DescState get') return self.value 10 def set(self, instance, value): # On attr assign print('DescState set') self.value = value class CalcAttrs: # Client class X = DescState(2) # Descriptor class attr Y = 3 # Class attr def init(self): self.Z = 4 # Instance attr obj = CalcAttrs() print(obj.X, obj.Y, obj.Z) # X is computed, others are not obj.X = 5 # X assignment is intercepted CalcAttrs.Y = 6 # Y reassigned in class obj.Z = 7 # Z assigned in instance print(obj.X, obj.Y, obj.Z) obj2 = CalcAttrs() # But X uses shared data, like Y!
```

`print(obj2.X, obj2.Y, obj2.Z)` This code's internal value information lives only in the descriptor, so there won't be a collision if the same name is used in the client's instance. Notice that only the descriptor attribute is managed here—get and set accesses to X are intercepted, but accesses to Y and Z are not (Y is attached to the client class and Z to the instance). When this code is run, X is computed when fetched, but its value is also the same for all client instances because it uses descriptor-level state:

```
$ python3 desc-state-desc.py DescState get 20 3 4 DescState set DescState get 50 6 7 DescState get 50 6 4
```

中文翻译

示例 38-6 把信息附在描述符自身实例上，因此不与客户端实例冲突——但也在两个客户端实例间共享该信息（代码与输出见英文原文）。内部 value 只活在描述符里，客户端实例即使用同名也不会撞车。这里只管理描述符属性：X 的读写被拦截，Y/Z 不被管理。因使用描述符级状态，所有客户端实例看到的 X 相同——obj2.X 也是 50。

深度理解

- 核心概念：obj.X 与 obj2.X 共享 DescState 上的 value——像类属性 Y 一样共享。
- 底层实现：CalcAttrs.X 只有一个 DescState(2) 对象。
- 设计原因：演示共享状态的利（无命名冲突）与弊（多实例串数据）。
- 实际问题：全局开关、共享缓存键（明确需要共享时）。
- 初学者误区：看到“不撞名”就永远用描述符状态——忘记业务数据常需 per-instance。

代码分析

```
#  
# obj:  X=20,Y=3,Z=4 → set X=5 → X=50,Y=6,Z=7  
# obj2: X=50, Y=6, Z=4
```

38.18 实例状态示例 (desc-state-inst.py)

英文原文

It's also feasible for a descriptor to store or use an attribute attached to the client class's instance instead of itself. Crucially, unlike data stored in the descriptor itself, this allows for data that can vary per client class instance. The descriptor in Example 38-7 assumes the instance has an attribute `X` attached by the client class and uses it to compute the value of the attribute it represents.

Example 38-7. desc-state-inst.py

```
python class InstState: # Using instance state
def get(self, instance, owner):
    print('InstState get')
    return instance.X
10 # Assume set by client class
def set(self, instance, value):
    print('InstState set')
    instance.X = value

class CalcAttrs:
    X = InstState() # Descriptor class
    attr Y = 3 # Class attr
    def init(self):
        self.X = 2 # Instance attr
    self.Z = 4 # Instance attr
obj = CalcAttrs()
print(obj.X, obj.Y, obj.Z)
obj.X = 5
CalcAttrs.Y = 6
obj.Z = 7
print(obj.X, obj.Y, obj.Z)
obj2 = CalcAttrs() # But X differs now, like Z!
```

`print(obj2.X, obj2.Y, obj2.Z)` Here, `X` is assigned to a descriptor as before that manages accesses. The new descriptor here, though, has no information itself, but it uses an attribute assumed to exist in the instance—

that attribute is named X, to avoid collisions with the name of the descriptor itself. When this version is run, the results are similar, but the value of the descriptor attribute can vary per client instance due to the differing state policy:

```
$ python3 desc-state-inst.py InstState get 20 3 4 InstState set InstState get 50 6 7 InstState get 20 6 4
```

Both descriptor and instance state have roles. In fact, this is a general advantage that descriptors have over properties—because they have state of their own, they can easily retain data internally without adding it to the namespace of the client instance object.

As a summary, the following uses both state sources—its `self.data` retains per-attribute information, while its `instance.data` can vary per client instance:

```
python >> class DescBoth:
... def init(self, data): ... self.data = data ... def get(self,
instance, owner): ... return f'{self.data}, {instance.data}'
'... def set(self, instance, value): ... instance.data = value ... >> class Client:
... def init(self, data): ... self.data = data ... managed = DescBoth('hack') ... >> I = Client('code') >> I.managed
d'hack, code'>> I.managed = 'HACK'>> I.managed'hack, HACK'
```

We'll revisit the implications of this choice in a case study later in this chapter. Before we move on, recall

from Chapter 32's coverage of slots that we can access “virtual” attributes like properties and descriptors with tools like `dir` and `getattr`, even though they don't exist in the instance's namespace dictionary.

Whether you should access these this way probably varies per program—properties and descriptors may run an arbitrary computation and may be less obviously instance “data” than slots:

```
python >> l.dict {'data':'HACK'} >> [x for x in dir(l) if
not x.startswith('_')] ['data','managed'] >> getattr(l,'da
ta'), getattr(l,'managed') ('HACK','hack, HACK')
```

The more generic `getattr` and `getattribute` tools we'll explore soon are not designed to support this functionality: because they have no class-level attributes, their “virtual” attribute names do not appear in `dir` results (per Chapter 31, a `dir` can provide a `dir` result, but it's optional and uncommon).

In exchange, they are also not limited to specific attribute names coded as properties or descriptors—tools that share even more than this behavior, as the next section explains.

中文翻译

描述符也可存储或使用附着在客户端类实例上的属性，而非自身。关键在于：与存在描述符内的数据不同，这允许数据随客户端类实例而变。示例 38-7 假定实例有客户端类附着的 `X`，并用它计算所代表属性的值（代码与输出见英文原文

）。此处 X 仍赋给管理访问的描述符，但描述符自身无信息，只用实例上假定存在的 X（前导下划线避免与描述符名冲突）。因状态策略不同，描述符属性值可随客户端实例变化：obj.X 为 20，不再与 obj 串台。两种状态都有角色。描述符相对 property 的一般优势是：有自有状态，可在内部保留数据而不塞进客户端实例命名空间。小结示例 Desc Both 同时使用两源：self.data 保留 per-attribute 信息，instance.data 可随客户端实例变化（见英文原文 REPL）。后文案例会再访这一选择的含义。另：第 32 章 slots 已提示，property/描述符这类“虚拟”属性即使用 dir/getattr 也能访问，即使不在实例 dict。是否应如此访问因程序而异——它们可能运行任意计算，不像 slots 那样明显是实例“数据”。即将探索的更通用的 getattr / getattribute 不为此设计：因无类级属性，其“虚拟”名默认不出现在 dir 中（可用 dir，但可选且少见）。作为交换，它们也不限于写成 property/描述符的特定属性名。

深度理解

- 核心概念：X 在实例、X 是描述符门面——经典 per-instance 模式。
- 底层实现：InstState 无 init 状态；一切经 instance。
- 设计原因：业务数据随对象变；描述符只提供行为。
- 实际问题：可出现在 dir/getattr 中，利于通用工具（序列化需小心触发计算）。
- 初学者误区：以为 getattr 管理的虚属性也会自动进 dir——不会。

代码分析

```
class InstState:
    def __get__(self, instance, owner):
        return instance._X * 10
    def __set__(self, instance, value):
        instance._X = value
```

· obj2 新建后 X=2, 故 X=20; 与描述符状态版形成对照实验。

38.19 How Properties and Descriptors Relate (property 与描述符如何关联)

英文原文

As mentioned earlier, properties and descriptors are strongly related—the property built-in is just a convenient way to create a descriptor. Now that you know how both work, you should also be able to see that it's possible to simulate the property built-in with a descriptor class, as demoed by Example 38-8.

Example 38-8. prop-desc-equiv.py

```
python class Property:
    def init(self, fget=None, fset=None, fdel=None, doc=None):
        self.fget = fget
        self.fset = fset
        self.fdel = fdel
        # Save unbound methods
        self.doc = doc
        # or other callables
        def get(self, instance, instancetype=None):
            if instance is None:
                return self
            if self.fget is None:
                raise AttributeError("can't get attrib
```

```

ute") return self.fget(instance) # Pass instance to self
in property accessors def set(self, instance, value): if s
elf.fset is None: raise AttributeError("can't set attribut
e") self.fset(instance, value) def delete(self, instance):
if self.fdel is None: raise AttributeError("can't delete a
ttribute") self.fdel(instance) class Person: def getNam
e(self): print('getName...') def setName(self, value): pr
int('setName...') name = Property(getName, setName
) # Use like property() x = Person() x.name
x.name = 'Pat' del x.name

```

This Property class catches attribute accesses with the descriptor protocol and routes requests to functions or methods passed in and saved in descriptor state when the class's instance is created. Attribute fetches, for example, are routed from the Person class, to the Property class's get method, and back to the Person class's getName. With descriptors, this "just works" :

```

$ python3 prop-desc-equiv.py getName... setName...
AttributeError: can't delete attribute

```

Note that this descriptor class equivalent only handles basic property usage, though; to use @ decorator syntax to also specify set and delete operations, we'd have to extend our Property class with setter and deleter methods, which would save the decorated accessor function and return the property object (self should suffice). Since the property built-in already does th

is, we'll omit a formal coding of this extension here.

中文翻译

如前所述，property 与描述符关系密切——property 内建只是创建描述符的便捷方式。既然两者都懂，应能看出可用描述符类模拟 property，如示例 38-8（完整代码见英文原文）。该 Property 类用描述符协议捕获访问，并把请求路由到创建时传入、保存在描述符状态中的函数/方法。例如读取路径：Person → Property.get → Person.getName。描述符下“就是能工作”。输出：getName... / setName... / 删除时 AttributeError。注意：该等价类只处理基础用法；若要用 @ 语法指定 set/delete，需扩展 setter/deleter 方法（保存装饰的访问器并返回 property 对象，self 即可）。因内建 property 已做这些，此处省略正式扩展编码。

深度理解

- 核心概念：property ≈ 保存 fget/fset/fdel 的数据描述符。
- 底层实现：自造 Property 与内建行为同构；内建还实现了 getter/setter/deleter 链式 API。
- 设计原因：教学上揭开“语法糖”的底层，建立统一心智模型。
- 实际问题：理解 Django 的 models.Field、SQLAlchemy 的 Column 为何像“超级 property”。
- 初学者误区：以为 property 与描述符是两套无关机制。

代码分析

```
def __get__(self, instance, instancetype=None):
    if instance is None:
        return self
    if self.fget is None:
        raise AttributeError("can't get attribute")
    return self.fget(instance)
```

- instance is None 时返回描述符自身（支持 Class.name 检查）。
- fget 为 None 时拒绝读取——对应 property(fset=...) 无 getter 的情形。

38.20 Descriptors and slots and more (描述符、slots 及更多)

英文原文

You can also probably now, at least in part, imagine how descriptors are used to implement Python's slots extension: instance attribute dictionaries are avoided by creating class-level descriptors that intercept slot name access and map those names to sequential storage space in the instance. Unlike the explicit property call, though, much of the magic behind slots is orchestrated at class creation time both automatically and implicitly when a slots attribute is present in a class. See Chapter 32 for more on slots—and why they're not recommended except in pathological use cases.

Descriptors are also used for other class tools, but we'll omit further internal details here; see Python's manuals and its open source code for more details. NOTE Descriptor cliff-hangers: In Chapter 39, we'll also make use of descriptors to implement function decorators that apply to both functions and methods. As you'll see there, because descriptors receive both descriptor and subject class instances they work well in this role, though nested functions are often a conceptually simpler solution. In addition, Chapter 39 deploys descriptors as one way to intercept built-in operation method fetches, and Chapter 40 formalizes data descriptors' precedence in the full inheritance model noted earlier: with a set, descriptors override other names and are thus fairly binding—they cannot be hidden by names in instance dictionaries.

中文翻译

你现在至少能部分想象：Python 的 slots 扩展如何用描述符实现——通过创建类级描述符拦截 slot 名访问，并把这些名字映射到实例中的顺序存储空间，从而避免实例属性字典。与显式 property 调用不同，slots 的大量魔法在类创建时、当类中存在 slots 属性时自动隐式编排。slots 详见第 32 章——以及为何除病态用例外不推荐。描述符还用于其他类工具，此处省略更多内部细节；见 Python 手册与开源代码。注描述符悬念：第 39 章会用描述符实现同时适用于函数与方法的函数装饰器——因描述符同时收到描述符与主体类实例，此角色很合适，不过嵌套函数常在概念上更

简单。第 39 章还会用描述符拦截内建运算方法读取；第 40 章将形式化前文所述数据描述符在完整继承模型中的优先级：带 set 时描述符覆盖其他名字，约束很强——不能被实例字典中的名字隐藏。

深度理解

- 核心概念：slots = 编译/创建期自动生成的数据描述符 + 固定布局存储。
- 底层实现：无 dict 时，属性访问全部走 slot 描述符。
- 设计原因：省内存、稍加速属性访问；代价是灵活性下降。
- 实际问题：大量小对象（是书中“病态”之一）才考虑 slots。
- 初学者误区：到处加 slots 当“优化”——过早优化且易与多重继承、默认值冲突。

38.21 getattr and getattribute (getattr 与 getattribute)

英文原文

So far, we've studied properties and descriptors—tools for managing specific attributes. The `getattr` and `getattribute` operator-overloading methods provide still other ways to intercept attribute fetches for class instances. Like properties and descriptors, they allow us to insert code to be run automatically when attribut

es are accessed. As shown here, though, these two methods can also be used in more general ways. Because they intercept arbitrary names, they can apply in broader roles, but may also incur extra calls in some contexts, and are too dynamic to register in dir results without help. This form of attribute-fetch interception comes in two flavors, coded with two different methods:- `getattr` is run for undefined attributes—because it is run only for attributes not stored on an instance or inherited from one of its classes, its use is straightforward.- `getattribute` is run for every attribute—because it is all-inclusive, you must be cautious when using this method to avoid recursive loops by passing attribute accesses to a superclass. We met the first of these in Chapter 30. These two methods are representatives of a set of attribute interception methods that also includes `setattr` and `delattr`. Because these methods have similar roles, though, we will generally treat them all as a single topic here. Unlike properties and descriptors, these methods are usually considered part of Python's operator-overloading protocol—specially named methods of a class, inherited by subclasses, and run automatically when instances are used in the associated built-in operation (here, attribute fetch). Like all normal methods of a class, they each receive a first `self` argument when called, giving access to both instance state information and other methods of their hosting class. The `getattr` and `getattribute` meth

ods are also more generic than properties and descriptors—they can be used to intercept access to any (or even all) instance attribute fetches, not just a single specific name. Because of this, these two methods are well suited to general delegation coding patterns—they can implement wrapper (a.k.a. proxy) objects that manage all attribute accesses for an embedded object. By contrast, we must define one property or descriptor for every attribute we wish to intercept. As covered ahead, this delegation role is limited somewhat for built-in operations but still applies to all named methods in a wrapped object's interface. Finally, these two methods are more narrowly focused than the alternatives we considered earlier: they intercept attribute fetches only, not assignments. To also catch attribute changes by assignment, we must code a `setattr` method—an operator-overloading method run for every attribute assignment, which must take care to avoid recursive loops by routing attribute assignments through the instance namespace dictionary or a superclass method. Although less common, we can also code a `delattr` overloading method (which must avoid looping in the same way) to intercept attribute deletions. By contrast, properties and descriptors catch `get`, `set`, and `delete` operations by design.

中文翻译

目前我们研究了 `property` 与描述符——管理特定属性的工具。运算符重载方法 `getattr` 与 `getattribute` 提供拦截类实例属性读取的另一些方式。与 `property`/描述符一样，它们允许在访问时自动运行代码；但如下所示，这两者也能以更通用的方式使用。因为它们拦截任意名字，可扮演更广角色，但某些上下文可能产生额外调用，且过于动态，若不加帮助不会登记在 `dir` 结果中。这种属性读取拦截有两种味道，由两个不同方法编码：- `getattr`：仅对未定义属性运行——只在属性未存在于实例、也未从任何类继承时运行，用法直接。- `getattribute`：对每一个属性运行——因包罗万象，使用时必须谨慎，通过把属性访问交给超类来避免递归循环。前者在第 30 章见过。二者代表还包括 `setattr`、`delattr` 的一组属性拦截方法；因角色相近，本章大体把它们当作同一主题。与 `property`/描述符不同，这些方法通常视为 Python 运算符重载协议的一部分——类的特殊命名方法，被子类继承，在实例用于相关内建操作（此处为属性读取）时自动运行。与所有普通方法一样，调用时收到第一个 `self`，可访问实例状态与宿主类其他方法。`getattr` / `getattribute` 也比 `property`/描述符更通用——可拦截任意（甚至全部）实例属性读取，而非仅单个特定名。因此二者很适合通用委托（`delegation`）模式——可实现包装/代理对象，管理嵌入对象的所有属性访问。相比之下，要对每个想拦截的属性各定义一个 `property` 或描述符。后文会讲：该委托角色对内建运算有所限制，但仍适用于被包装对象接口中的所有具名方法。最后，这两者比先前方案更“窄”：只拦截读取，不拦截赋值。要捕获赋值，须编写 `setattr`——对每次属性赋值运行的运算符重载方法，且须通过实例命名空间字典或超类方法路由赋值以避免递归。较少见地，也可编

写 `delattr` (同样须避免循环) 拦截删除。相比之下, `property` 与描述符在设计上就同时捕获 `get/set/delete`。

深度理解

- 核心概念: 通用拦截 vs 具名拦截——代理/委托选前者, 单字段校验选后者。
- 底层实现: `getattr` 是查找失败后的钩子; `getattribute` 是查找入口钩子。
- 设计原因: 委托模式 (第 31 章) 需要 “任意名字转发”, `property` 做不到一个搞定。
- 实际问题: 远程代理、ORM 懒加载、动态 API 包装。
- 初学者误区: 用 `getattribute` 做只拦一个字段的事——又慢又易递归。

38.22 The Basics (`getattr` / `getattribute` 基础)

英文原文

In short, if a class defines or inherits the following methods, they will be run automatically when an instance is used in the operation described by the comments to the right: `python def getattr(self, name): # On undefined attribute fetch [obj.name]` `def getattribute(self, name): # On all attribute fetch [obj.name]` `def setattr(self, name, value): # On all attribute assignment [obj.name=value]` `def delattr(self, name): # On all attrib`

ute deletion [del obj.name] In these, self is the subject instance object as usual, name is the string name of the attribute being accessed, and value is the object being assigned to the attribute. The two get methods normally return an attribute's value, and the other two return nothing (None). All can raise exceptions to signal prohibited access. For example, to catch every attribute fetch, we can use either of the first two previous methods, and to catch every attribute assignment we can use the third. The following uses getattr for fetches:

```

python class Catcher:
    def getattr(self, name):
        print('Get:', name)
    def setattr(self, name, value):
        print('Set:', name, value)
X = Catcher()
X.job # Prints "Get: job"
X.pay # Prints "Get: pay"
X.pay = 'bread' # Prints "Set: pay bread"

```

Using getattribute works exactly the same in this specific case but has subtle looping potential which we'll take up in the next section. Such a coding structure can be used to implement the delegation design pattern we met earlier in Chapter 31. Because all attributes are routed to interception methods generically, we can validate and pass them along to embedded, managed objects. As a refresher, the following class, borrowed from Chapter 31, traces every attribute fetch made to another object passed to the wrapper (proxy) class:

```

python class Wrapper:
    def init(self, object):
        self.wrapped = object # Save object
    def getattr(self, attrname):
        print('Trace:', attrname) # Trace fetch
        return getattr(self.wrapped, attrname) # Dele

```

```
gate fetch X = Wrapper([1, 2, 3]) X.append(4) # Prints  
"Trace: append" print(X.wrapped) # Prints "[1, 2, 3, 4]"
```

There is no such analog for properties and descriptors, short of coding accessors for every attribute present in every wrapped object. On the other hand, when such generality is not required, generic accessor methods may incur additional calls for assignments in some contexts—a trade-off described in Chapter 30 and mentioned in the context of the case study example we'll explore at the end of this chapter.

中文翻译

简言之，若类定义或继承下列方法，当实例用于注释所述操作时它们会自动运行（见英文原文四方法签名）。其中 `self` 为主体实例，`name` 为被访问属性的字符串名，`value` 为赋给属性的对象。两个 `get` 方法通常返回属性值，另两个返回 `None`。都可抛异常表示禁止访问。例如要捕获每次读取可用前两个之一，捕获每次赋值用第三个。`Catcher` 示例用 `getattr` 处理读取。同一情形用 `getattribute` 效果相同，但有微妙的循环风险（下一节讨论）。这种结构可用于实现第 31 章的委托设计模式。因所有属性泛化路由到拦截方法，我们可校验并转发给嵌入的被管理对象。复习：从第 31 章借来的 `Wrapper` 追踪对传入包装（代理）类的另一对象的每次属性读取。`property`/描述符没有这种对等物——除非为被包装对象的每个属性各写访问器。另一方面，若不需要这种通用性，通用访问器方法在某些上下文中可能对赋值产生额外调用——这是第 30 章描述、本章末案例也会提到的权衡。

深度理解

- 核心概念：四方法构成“属性运算符重载”全家桶。
- 底层实现：X.job 若未定义 → getattr(X,'job'); X.pay = ... → setattr。
- 设计原因：Wrapper 一行 getattr 即可代理任意方法——这是动态语言优势。
- 实际问题：日志代理、远程 RPC 桩、测试 mock。
- 初学者误区：getattr 里 return self.wrapped.attrname 用点号会再查——应 getattr(self.wrapped, attrname)。

代码分析

```
class Wrapper:
    def __init__(self, object):
        self.wrapped = object
    def __getattr__(self, attrname):
        print('Trace:', attrname)
        return getattr(self.wrapped, attrname)

X = Wrapper([1, 2, 3])
X.append(4) # Trace: append list.append
```

- append 不在 Wrapper 实例/类上 → 进 getattr → 返回真正的绑定方法 → 调用之。

38.23 Avoiding loops in attribute interception methods（避免属性拦截方法中的循环）

英文原文

These methods are generally straightforward to use. Their most complex aspect is the potential for looping (a.k.a. recursing). Because `getattr` is called for undefined attributes only, it can freely fetch other attributes within its own code. However, because `getattr` and `setattr` are run for all attributes, their code must be careful when accessing other attributes to avoid calling themselves again and triggering a recursive loop. For example, another attribute fetch run inside a `getattr` method's code like the following will trigger `getattr` again—and the code will usually loop until memory is exhausted: `python def getattr(self, name): x = self.other # LOOPS!` Technically, this method is even more loop-prone than this may imply—a self attribute reference run anywhere in a class that defines this method will trigger `getattr` and also has the potential to loop, depending on the class's logic. This is normally desired behavior—intercepting every attribute fetch is this method's purpose, after all—but you should be aware that this method catches all attribute fetches wherever they are coded. When coded within `getattr` itself, this almost always causes a loop. To avoid this loop, route the fetch through a higher superclass instead to skip this level's version—because the object class is always a superclass to every class, it serves well in this role: `python def ge`

`ttribute(self, name): x = object.getattribute(self, 'other') # Force higher to avoid me` For `setattr`, the situation is similar—assigning any attribute inside this method triggers `setattr` again and may create a similar loop:

```
python def setattr(self, name, value): self.other = value # Recurs (and might LOOP!) Here too, self attribute assignments anywhere in a class defining this method trigger setattr as well, though the potential for looping is much stronger when they show up in setattr itself. To work around this problem, you can assign the attribute as a key in the instance's dict namespace dictionary instead. This avoids direct attribute assignment: python def setattr(self, name, value): self.dict['other'] = value # Use attr dict to avoid me
```

Alternatively, `setattr` can also pass its own attribute assignments to a higher superclass to avoid looping, just like `getattribute`. In fact, this scheme is sometimes preferred when wrapped classes use slots, properties, or other “virtual” attributes that live on classes instead of instances—and in the case of slots, may preclude dict:

```
python def setattr(self, name, value): object.setattr(self, 'other', value) # Force higher to avoid me
```

This book's `setattr` examples often use dict for smaller demos anyway, just because their parameters are known. By contrast, though, we cannot use the dict trick to avoid loops in `getattribute`:

```
python def getattribute(self, name): x = self.dict['other'] # Loops! If this is coded, fetching the dict attribute itself triggers getattribute ag
```

ain—causing a recursive loop and an immediate fail. Strange but true! The `delattr` method is less commonly used in practice, but when it is, it is called for every attribute deletion, just as `setattr` is called for every attribute assignment. When using this method, you must avoid loops when deleting attributes by the same techniques: namespace dictionaries operations or superclass method calls.

中文翻译

这些方法通常好用。最复杂的是循环/递归风险。`getattr` 只对未定义属性调用，可在自身代码中自由读取其他（已定义）属性。但 `getattribute` 与 `setattr` 对所有属性运行，访问其他属性时必须小心，以免再次调用自身触发递归循环。例如在 `getattribute` 内写 `x = self.other` 会再次触发自身——通常循环到内存耗尽。技术上，定义了该方法的类中任何地方的 `self` 属性引用都会触发它，是否循环取决于类逻辑。这通常是期望行为——拦截每次读取正是其目的——但须意识到：无论写在何处，它都捕获所有读取；写在 `getattribute` 自身内部则几乎总是循环。避免办法：通过更高超类路由读取以跳过本层——`object` 总是每类的超类，很适合：`x = object.getattribute(self, 'other')`。`setattr` 类似：方法内 `self.other = value` 会再次进入并可能循环。变通：写入实例 `dict` 键，避免直接属性赋值；或 `object setattr(self, 'other', value)`。当包装类使用 `slots/property` 等“虚拟”属性（且 `slots` 可能没有 `dict`）时，超类方案有时更优。本书小演示常用 `dict`。但 `getattribute` 不能用 `dict` 技巧避循环——因为取 `self.dict` 本身也会再进 `getattribute`

bute。奇怪但属实！delattr 较少用，但对每次删除调用；
避循环手法相同。

深度理解

- 核心概念：全拦截方法的第一纪律——永远不要在内部用 self.xxx 点号访问（除非你清楚不会再入）。
- 底层实现：object.getattribute / object.setattr 是 C 层实现，不经过你的 Python 覆盖。
- 设计原因：把“默认属性机器”留在 object，用户覆盖时显式“升级调用”。
- 实际问题：slots 无 dict 时必须用 object.setattr。
- 初学者误区：在 getattribute 里 return self.dict[name]——立刻爆栈。

代码分析

```
#
def __getattribute__(self, name):
    x = self.other                #
    x = self.__dict__['other']    #

#
def __getattribute__(self, name):
    x = object.__getattribute__(self, 'other')

def __setattr__(self, name, value):
    self.__dict__[name] = value
    # object.__setattr__(self, name, value)
```

38.24 A First Example (getattr 第一个示例)

英文原文

Generic attribute management is not nearly as complicated as the prior section may have implied. To see how to put these ideas to work, Example 38-9 is the same first example we used for properties and descriptors in action again, this time implemented with attribute operator-overloading methods. Because these methods are so generic, we test attribute names here to know when a managed attribute is being accessed ; others are allowed to pass normally. Example 38-9.

```
getattr-person.py python class Person: def init(self, name): # On [Person()] self.name = name # Triggers setattr! def getattr(self, attr): # On [obj.undefined] print('get:' + attr) if attr == 'name': # Intercept name: not stored return self.name # Does not loop: real attr else: # Others are errors raise AttributeError(attr) def setattr(self, attr, value): # On [obj.any = value] print('set:' + attr) if attr == 'name': attr = 'name' # Set internal name self.dict[attr] = value # Avoid looping here def delattr(self, attr): # On [del obj.any] print('del:' + attr) if attr == 'name': attr = 'name' del self.dict[attr] # Avoid looping here too sue = Person('Sue Jones') print(sue.name) # Runs getattr sue.name = 'Susan Jones' # Runs setattr print(sue.name) del sue.name # Runs delattr
```

```
print('-'20) bob = Person('Bob Smith') print(bob.name) # print(Person.name.doc) # No direct equivalent here!
```

When this code is run, the same sort of output is produced, but this time it reflects our generic attribute-interception methods responding to Python's normal operator-overloading mechanism. Notice how the attribute assignment in the `init` constructor triggers `setattr` too—this method catches every instance-attribute assignment, even those anywhere within the class itself, and those to underlying attributes like `name`. Also note that, unlike with properties and descriptors, there's no direct notion of specifying documentation for our attribute here; managed attributes exist within the code of our interception methods, not as distinct objects.

中文翻译

通用属性管理远没有上一节暗示的那么复杂。示例 38-9 再次实现 `property`/描述符的同一用例，这次用属性运算符重载。因方法过于通用，这里通过测试属性名判断何时在访问受管属性；其他名字正常通过（代码见英文原文）。运行产生同类输出，但反映的是通用拦截方法响应运算符重载机制。注意：`init` 中的赋值也会触发 `setattr`——它捕获每一次实例属性赋值，包括类内部、以及对 `name` 等底层属性的赋值。另：与 `property`/描述符不同，这里没有为属性指定文档的直接概念；受管属性存在于拦截方法的代码中，而非独立对象。

深度理解

- 核心概念：用字符串名分支模拟“具名 property”；name 不存实例上，故读取走 getattr。
- 底层实现：self.name = name 进 setattr，经 dict 写入避免循环；读 name 因未定义进 getattr。
- 设计原因：展示通用机制也能做单字段管理，但不如 property 干净。
- 实际问题：动态属性名、运行时才知道的字段集。
- 初学者误区：在 getattr 里 return self.name 会再入（若 name 仍未定义）——应读 name。

代码分析

```
def __setattr__(self, attr, value):  
    if attr == 'name':  
        attr = '_name'  
    self.__dict__[attr] = value
```

- 把对外名 name 映射到对内名 name，统一入口做变换/校验。

38.25 Using getattribute (改用 getattribute)

英文原文

To achieve exactly the same results with getattribute, replace getattr in Example 38-9 with the differing co

de in Example 38-10. Because it catches all attribute fetches, this version must be careful to avoid looping by passing new fetches to a superclass, and it can't generally assume unknown names are errors. Example 38-10. `getattribute-person.py` (differing part) python

```
def getattribute(self, attr): # On [obj.any] print('get:' + attr) if attr == 'name': # Intercept all names attr = 'name' # Map to internal name return object.getattribute(self, attr) # Avoid looping here
```

When run with this change, the output is similar, but we get an extra `getattribute` call for the fetch of `dict` in `setattr` (the first time originating in `init`). This example is equivalent to that coded for properties and descriptors, but it's a bit artificial, and it doesn't really highlight these tools' assets. Because they are generic, `getattr` and `getattribute` are probably more commonly used in delegation-based code (as sketched earlier), where attribute access is validated and routed to an embedded object. Where just a single attribute must be managed, properties and descriptors might do as well or better, and avoid extra calls for unmanaged attributes.

中文翻译

要用 `getattribute` 得到完全相同结果，把示例 38-9 的 `getattr` 换成示例 38-10 的差异代码。因捕获所有读取，此版本须把新读取交给超类以避免循环，且一般不能假定未知名就是错误。运行输出类似，但 `setattr` 里取 `dict` 时会多一次 `getattribute` 调用（首次源自 `init`）。本例与 `property`/描

述符版等价，但有些造作，并未真正凸显这些工具的长处。因其通用，`getattr` / `getattribute` 更常用于基于委托的代码，在那里校验访问并路由到嵌入对象。若只需管理单个属性，`property`/描述符可能同样好或更好，且对未管理属性避免额外调用。

深度理解

- 核心概念：全拦截必须“默认转发”到 `object.getattribute`。
- 底层实现：`self.dict[attr] = value` 中的 `self.dict` 也走 `getattribute`。
- 设计原因：一致性——一切读取可见；代价是噪音与性能。
- 实际问题：单字段场景优先 `property`。
- 初学者误区：对未知名 `raise AttributeError` 而不转发——会弄坏正常属性。

38.26 Computed Attributes (`getattr` 计算属性)

英文原文

As before, our prior example doesn't really do anything but trace attribute fetches; it's not much more work to compute an attribute's value when fetched. As for properties and descriptors, Example 38-11 creates a virtual attribute `X` that runs a calculation when fetched

hed. Example 38-11. `getattr-computed.py`

```
python class AttrSquare: def init(self, start): self.value = start # Triggers setattr! def getattr(self, attr): # On undefined attr fetch if attr == 'X': return self.value 2 # value is not undefined else: raise AttributeError(attr) def setattr(self, attr, value): # On all attr assignments if attr == 'X': attr = 'value' self.dict[attr] = value A = AttrSquare(3) B = AttrSquare(32) print(A.X) # 3 2 => 9 A.X = 4 print(A.X) # 4 2 => 16 print(B.X) # 32 2 => 1024
```

Running this code results in the same output that we got earlier when using properties and descriptors, but this script's mechanics are based on generic attribute interception methods.

中文翻译

与前一样，上例主要只是追踪读取；在读取时计算属性值并不多费功夫。示例 38-11 创建虚拟属性 X，读取时运行计算。输出仍为 9、16、1024，但机制基于通用属性拦截方法。

深度理解

- 核心概念：X 从不存实例上 → 每次读都进 `getattr`；底层数据在 `value`。
- 底层实现：`self.value` 在 `getattr` 内是已定义属性，不递归。
- 设计原因：虚属性 + 实存储的经典拆分。
- 实际问题：与 `property` 计算属性同构，写法更“过程式”

”。

·初学者误区：把结果缓存进 self.X 后，getattr 再也不会被调用（有时是特性，有时是 bug）。

代码分析

```
def __getattr__(self, attr):
    if attr == 'X':
        return self.value ** 2 # value __getattr__
```

38.27 Using getattribute for computed attributes (计算属性的 getattribute 版)

英文原文

As before, we can achieve the same effect with getattribute instead of getattr. Example 38-12 replaces the fetch method with a getattribute and changes the setattr assignment method to avoid looping by using direct object superclass method calls instead of dict keys. Example 38-12. getattribute-computed.py python class AttrSquare: def init(self, start): self.value = start def getattribute(self, attr): if attr == 'X': return self.value ** 2 # Triggers getattribute again! else: return object.getattribute(self, attr) def setattr(self, attr, value): if attr == 'X': attr = 'value' object.setattr(self, attr, value)

When this version is run, the results are the same again. Notice, though, the implicit and subtle routing: self.value=start inside the constructor triggers setattr; s

self.value inside getattribute triggers getattribute again. In fact, getattribute is run twice each time we fetch attribute X. This doesn't happen in the getattr version because the value attribute is not undefined. If you care about speed, change getattribute to use the superclass to fetch value as well: python def getattribute(self, attr): if attr == 'X': return object.getattribute(self, 'value') 2

中文翻译

同样可用 getattribute 达到相同效果。示例 38-12 用 getattribute 替换读取方法，并用 object 超类方法调用避免 setattr 循环。结果再次相同。但注意：构造器中 self.value = start 触发 setattr；getattribute 内的 self.value 再次触发 getattribute。事实上，每次读取 X 时 getattribute 会跑两次。getattr 版不会，因为 value 不是未定义的。若在意速度，对 value 也走超类读取。

深度理解

- 核心概念：getattribute 版“看起来能跑”仍可能有隐藏的双倍调用。
- 底层实现：self.value → 再入 → attr != 'X' 分支 → object.getattribute。
- 设计原因：教学揭示全拦截的隐藏成本。
- 实际问题：热路径上避免 getattribute 内的点号访问。
- 初学者误区：用 print 调试 getattribute 时 print(self.xx) 可能改变控制流或爆栈。

38.28 getattr and getattribute Compared (二者对比)

英文原文

To summarize the coding differences between `getattr` and `getattribute`, Example 38-13 uses both to implement three attributes—`attr1` is a class attribute, `attr2` is an instance attribute, and `attr3` is a virtual managed attribute computed when fetched. Example 38-13.

```
getattr-v-getattribute.py python class GetAttr: attr1 = 1 def init(self): self.attr2 = 2 def getattr(self, attr): print('get:', attr) if attr == 'attr3': return 3 else: raise AttributeError(attr) class GetAttribute: attr1 = 1 def init(self): self.attr2 = 2 def getattribute(self, attr): print('get:', attr) if attr == 'attr3': return 3 else: return object.getattribute(self, attr)
```

When run, the `getattr` version intercepts only `attr3` accesses because it is undefined. The `getattribute` version intercepts all attribute fetches and must route those it does not manage to the superclass fetcher to avoid loops. Output shows `attr1/attr2` silent in the first half, and `get:` for all three names in the second half. Although `getattribute` can catch more attribute fetches than `getattr`, in practice they are often just variations on a theme—if attributes are not physically stored, the two have the same effect.

中文翻译

为总结编码差异，示例 38-13 用二者各实现三属性：attr1 类属性，attr2 实例属性，attr3 读取时计算的虚拟受管属性。运行时，getattr 版只拦截未定义的 attr3；getattribute 版拦截所有读取，且须把未管理的交给超类以避免循环。虽 getattribute 能捕获更多，实践中二者常只是同一主题的变体——若属性未物理存储，效果相同。

深度理解

- 核心概念：一张对照表胜过千言——谁被拦截一目了然。
- 底层实现：已定义名跳过 getattr；永远不跳过 getattribute（除非走 object）。
- 设计原因：默认用 getattr（更安全）；只有需要“监视一切”时才用 getattribute。
- 实际问题：权限审计可能需要后者；缺省值/虚属性用前者。
- 初学者误区：两者混用时顺序/交互复杂——通常只选一个。

38.29 Management Techniques Compared (管理技术对比)

英文原文

To summarize the coding differences in all four attribute-management schemes we've just explored, let's q

quickly step through a somewhat more comprehensive computed-attribute example using each technique. Example 38-14. `allfourprops.py` — properties for square and cube, base values in square/cube. Example 38-15. `allfourdesc.py` — descriptors with per-instance state (`instance.square / instance.cube`). Example 38-16. `allfourgetattr.py` — `getattr` + `setattr`, underscore bases so managed names stay undefined on fetch. Example 38-17. `allfourgetattribute.py` — `getattribute` + `setattr`, route bases through `object.getattribute / object.setattribute`.

```
python # 38-14 sketch class Powers:
def init(self, square, cube):
    self.square = square
    self.cube = cube
def getSquare(self):
    return self.square ** 2
def setSquare(self, value):
    self.square = value
square = property(getSquare, setSquare)
def getCube(self):
    return self.cube ** 3
cube = property(getCube)
python # 38-15 sketch class DescSquare:
def get(self, instance, owner):
    return instance.square ** 2
def set(self, instance, value):
    instance.square = value
class DescCube:
def get(self, instance, owner):
    return instance.cube ** 3
class Powers:
square = DescSquare()
cube = DescCube()
def init(self, square, cube):
    self.square = square
    self.cube = cube
python # 38-16 sketch class Powers:
def init(self, square, cube):
    self.square = square
    self.cube = cube
def getattr(self, name):
    if name == 'square':
        return self.square ** 2
    elif name == 'cube':
        return self.cube ** 3
    else:
        raise TypeError('unknown attr:' + name)
def setattr(self, name, value):
    if name == 'square':
        self.dict['square'] = va
```

```
lue else: self.dict[name] = value python # 38-17 sketch
class Powers:
    def init(self, square, cube):
        self.square = square
        self.cube = cube
    def getattribute(self, name):
        if name == 'square':
            return object.getattribute(self, 'square')
        elif name == 'cube':
            return object.getattribute(self, 'cube')
        else:
            return object.getattribute(self, name)
    def setattr(self, name, value):
        if name == 'square':
            object.setattr(self, 'square', value)
        else:
            object.setattr(self, name, value)
Each technique takes a different form in code, but all four produce the same result when run (square/cube on bases 3 and 4 yield 9 and 64; after square=5, square is 25). For more realistic comparison, stay tuned for the attribute-validation example ahead. First, a short side trip: a pitfall of the generic attribute interceptors.
```

中文翻译

为总结四种属性管理方案的编码差异，我们用每种技术快速走一遍更全面的计算属性示例。

- 38-14 property: square/cube, 基值 square/cube
- 38-15 描述符: per-instance 状态
- 38-16 getattr + setattr
- 38-17 getattribute + setattr, 基值走 object.

四者代码形态不同，运行结果相同。更现实对比见后文校验案例；先旁路讨论通用拦截器的陷阱。

深度理解

·核心概念：同一语义，四种句法——选型看“特定 vs 通用、是否自带状态、调用开销、dir 可见性”。

- 底层实现：四路最终都是“读时计算、写时映射到存储名”。
- 设计原因：Python 多范式——工具构建者按场景选刀。
- 实际问题：团队规范常约定“单字段用 property，代理用 getattr”。
- 初学者误区：在同一类里叠用四种——调试地狱。

代码分析

四版对 Powers(3,4) 均应得到：square==9, cube==64, square=5 后 square==25。

38.30 Intercepting Built-in Operation Attributes (拦截内建运算属性)

英文原文

If you've been reading this book linearly, some of this section is elaboration on earlier notes, especially the sidebar “Delegating Built-ins—or Not”. When `getattr` and `getattribute` were introduced here, it was stated that they intercept undefined- and all-attribute fetches, respectively, which makes them ideal for delegation-based coding patterns. While this is true for both normally named and explicitly fetched attributes, their behavior needs some additional clarification. Specifically, the implicit method-name fetches of built-in operations will never automatically be routed to eit

her of these two attribute-interceptor methods. This means that operator-overloading method calls cannot be delegated to wrapped objects unless wrapper classes somehow redefine these methods themselves. For example, attribute fetches for the `str`, `add`, and `getitem` methods run implicitly by printing, `+` expressions, and indexing, respectively, are not routed to either `getattr` or `getattribute`. Instead, such methods are looked up in classes, and skip the instance and its attribute interceptors. Hence, there is no direct way to generically catch and delegate built-in operations like these. This was a Python 3.X bifurcation, whose purported rationale involved metaclasses and optimization of built-in operations. Whatever its basis, all attributes—both `X` and other—are still dispatched through the instance's interceptor methods when accessed explicitly by name, so this qualifies as a glaring inconsistency: `X.add` runs `getattr`, but `X + Y`, which uses `X.add`, does not. The net effect complicates delegation-based code. The good news is that wrapper classes can work around this constraint by redefining operator-overloading methods in the wrapper itself, in order to catch and delegate calls. These extra methods can be added either manually, with tools, or by definition in, and inheritance from, common superclasses. It's more work for delegation classes when operator-overloading methods are part of a wrapped object's interface, but it's not a showstopper.

中文翻译

若你线性阅读本书，本节有些是对早先注释的展开，尤其是边栏“Delegating Built-ins—or Not”。介绍 `getattr` / `getattrattribute` 时曾说它们分别拦截未定义与全部属性读取，因而理想适合委托模式。对普通命名与显式按名读取的属性，这确实成立；但对内建运算的隐式方法名读取，行为需要澄清：这些隐式读取永远不会自动路由到这两个拦截方法。这意味着：除非包装类自己重定义这些方法，否则运算符重载调用无法委托给被包装对象。例如，打印、`+`、索引分别隐式运行的 `str`、`add`、`getitem`，都不会进 `getattr` 或 `getattrattribute`。这类方法在类中查找，跳过实例及其属性拦截器。因此没有直接方式泛化捕获并委托这类内建运算。这是 Python 3.X 的分叉，声称理由涉及元类与内建运算优化。不论依据如何，当显式按名访问时，所有属性（含 X）仍会经过实例拦截方法——于是出现刺眼不一致：`X.add` 会跑 `getattr`，但使用 `X.add` 的 `X + Y` 却不会。净效果使委托代码更复杂。好消息：包装类可在自身重定义运算符重载方法以捕获并委托。这些额外方法可手工添加、用工具生成，或在公共超类中定义再继承。当运算符方法属于被包装接口时，委托类工作量更大，但并非死局。

深度理解

- 核心概念：3.X 中，内建运算找 X 从类开始，不走实例 `getattrattribute/getattr`。
- 底层实现：速度优化——避免每次 `+` 都进 Python 层拦截。

- 设计原因：与显式 `X.add` 路径不一致，是委托代理的经典坑。
 - 实际问题：代理 `list` 时 `len(X)`、`X[0]`、`print(X)` 可能“失灵”。
 - 初学者误区：以为写了 `getattr` 就自动支持所有运算符——`3.X` 不成立。
-

38.31 `getattr-builtins` 演示（内建运算演示）

英文原文

As a demo of the issue, consider the code in Example 38-18, which tests various attribute types and built-in operations on instances of classes containing `getattr` and `getattribute` methods. Example 38-18. `getattr-builtins.py` (abridged structure)

```
python class GetAttr:
    attr = 88
    def init(self):
        self.iattr = 77
    def len(self):
        print('len: 66')
        return 66
    def getattr(self, attr):
        print('getattr:', attr)
        if attr == 'str':
            return lambda args: '[Getattr str]'
        else:
            return lambda args: None
class GetAttribute:
    attr = 88
    def init(self):
        self.iattr = 77
    def len(self):
        print('len: 66')
        return 66
    def getattribute(self, attr):
        print('getattribute:', attr)
        if attr == 'str':
            return lambda args: '[GetAttribute str]'
        else:
            return lambda args: None
```

Neither `getattr` nor `getattribute` are run for any of the operator-overloading names invoked by built-in operations because such names are looked up in classes only. Salient points:- `str` access fails to be caught twice by

getattr: once for the built-in print, and once for explicit fetches because a default is inherited from the built-in object implied above every topmost class.- str fails to be caught only once by the getattrattribute catchall—during the built-in print operation. Explicit fetches bypass the inherited str and run getattrattribute.- call fails to be caught in both schemes for built-in call expressions, but it is intercepted by both when fetched explicitly; unlike str, there is no inherited call default in object to defeat getattr in explicit fetches. The same goes for the add of + operations.- len is handled by both classes because it is an explicitly defined method in the classes themselves—though its name is not routed to either interceptor if we delete the classes' len methods, because the len built-in skips them as usual. Again, the net effect is that operator-overloading methods implicitly run by built-in operations are never routed through either attribute interception method. Python begins the search for such attributes in classes and skips instance lookup mechanisms entirely. Normally, named attributes and explicit fetches start with the instance instead. For a more realistic example of this phenomenon's impact on delegation classes, stay tuned for Chapter 39's Private decorator—along with its coverage of multiple reusable workarounds.

中文翻译

作为问题演示，示例 38-18 在含 `getattr` 与 `getattrattribute` 的类实例上测试多种属性类型与内建运算（结构见英文原文）。对内建运算触发的任何运算符重载名，二者都不会运行——这些名字只在类中查找。要点：- `getattr` 对 `str` 两次都抓不到：一次因内建 `print`，一次因显式读取时从隐含的 `object` 继承了默认 `str`。- `getattrattribute` 只在内建 `print` 时抓不到 `str`；显式读取会绕过继承的 `str` 并进入 `getattrattribute`。- 二者对内建调用表达式都抓不到 `call`，但显式读取时都能拦截；与 `str` 不同，`object` 没有默认 `call` 会在显式路径上击败 `getattr`。+ 的 `add` 同理。- `len` 因在类中显式定义而被处理；若删除类的 `len`，`len` 内建仍会按惯例跳过拦截器。净效果：内建运算隐式运行的运算符方法从不经这两种属性拦截。Python 从类开始搜这类属性，完全跳过实例查找。通常，具名属性与显式读取从实例开始。更现实的影响见第 39 章 `Private` 装饰器及其多种可复用变通方案。

深度理解

- 核心概念：显式 `X.add(y)` $\neq$ 隐式 `X+y` 的属性查找路径。
- 底层实现：内建类型槽（`tpasnumber` 等）直接从类型对象取函数指针。
- 设计原因：性能；副作用是代理必须“补全”常用 `X`。
- 实际问题：第 39 章会给混入超类等修复模式。
- 初学者误区：在 `getattr` 返回 `str` 却发现 `print(obj)` 仍是默认 `repr`。

代码分析

```
# __getattr__ / __getattribute__
print(X)      # __str__ / __repr__
X + 99        # __add__
X[0]          # __getitem__

#
X.__add__(99)
X.__str__()
```

38.32 Revisiting Chapter 28's delegation example (重访第 28 章委托示例)

英文原文

As a coda, you should also now be able to work out why the Manager class coded in Example 28-11 of Chapter 28 had to code a repr to route printing requests to its wrapped object. Just like str in our demo, object provides a default repr, which would prevent print operations from invoking a getattr. Technically speaking, object defines both str and repr, but its str simply calls repr. That said, object's defaults are largely a moot point: like all built-in operations, print bypasses both getattr and getattribute, as it did for getattribute in our demo. Hence, a repr is required by both the object default and the built-in's behavior. Again, fixes for delegating built-ins are in Chapter 39 (unless we run out of underscores before that!).

中文翻译

作为尾声，你现在应能明白：第 28 章示例 28-11 的 Manager 为何必须自写 repr 把打印请求路由到被包装对象。正如演示中的 str，object 提供默认 repr，会阻止打印操作调用 getattr。技术上 object 同时定义 str 与 repr，但其 str 只是调用 repr。话说回来，object 的默认很大程度上已不重要：与所有内建运算一样，print 会绕过 getattr 与 getattribute。因此，无论是 object 默认还是内建行为，都需要自写 repr。委托内建的修复见第 39 章。

深度理解

- 核心概念：代理类常需显式转发 repr/str/iter 等。
- 底层实现：继承链上的 object.repr “占坑”，让 getattr 看不到 repr。
- 设计原因：每个对象都要能打印；默认实现保证这一点。
- 实际问题：调试时代理打印无用信息。
- 初学者误区：只测 obj.method()，不测 print/len/+，上线才发现委托残缺。

38.33 Example: Attribute Validations (示例：属性校验)

英文原文

To close out this chapter, let's turn to a more realistic example, coded in all four of our attribute managem

ent schemes. The example we will use defines a Card Holder object with four attributes, three of which are managed. The managed attributes validate or transform values when fetched or stored. All four versions produce the same results for the same test code, but they implement their attributes in very different ways. The examples are largely for self-study; although we won't go through their code in detail, they all use concepts we've already explored in this chapter.

中文翻译

为收束本章，我们转向更现实的示例，用全部四种属性管理方案编码。示例定义带四个属性的 CardHolder 对象，其中三个受管。受管属性在读取或存储时校验或变换值。四个版本对同一测试代码产生相同结果，但实现方式大不相同。示例 largely 供自学；虽不逐行细讲，但都使用本章已探索的概念。

深度理解

- 核心概念：同一业务规则（姓名规范化、年龄范围、账号掩码、只读剩余工龄）× 四种实现。
- 底层实现：差异在“钩子放哪、状态放哪、未管理属性是否额外调用”。
- 设计原因：对照实验是选技术的最佳教材。
- 实际问题：表单/领域对象校验、敏感字段脱敏。
- 初学者误区：只抄一种实现，不理解状态共享陷阱（描述符版 option 1）。

38.34 Using Properties to Validate (用 property 做校验)

英文原文

Our first coding in Example 38-19 uses properties to manage three attributes. As usual, we could use simple methods instead of managed attributes, but properties help if we have already been using attributes in existing code. Properties run code automatically on an attribute access but are focused on a specific set of attributes; they cannot be used to intercept all attributes generically. To understand this code, it's crucial to notice that the attribute assignments inside the `__init__` constructor method trigger property setter methods too. When this method assigns to `self.name`, for example, it automatically invokes the `setName` method, which transforms the value and assigns it to an instance attribute called `_name` so it won't clash with the property's name. This renaming, sometimes called *name mangling*, is important because properties use common instance state and have none of their own. Data is stored in an attribute called `_name`, and the attribute called `name` is always a property, not data. As we saw in Chapter 31, names like `_name` are known as *pseudoprivate* attributes and are changed by Python to include the enclosing class's name when stored in the instance.

e's namespace; here, this helps keep the implementation-specific attributes distinct from others, including that of the property that manages them. In the end, this class manages attributes called name, age, and acct; allows the attribute addr to be accessed directly; and provides a read-only attribute called remain that is entirely virtual and computed on demand. For comparison purposes, this property-based coding weighs in at 39 lines of code (including blank lines). Example 38-19. validateproperties.py

```
python class CardHolder:
    acctlen = 8 # Class data
    retireage = 62.5
    def init(self, acct, name, age, addr):
        self.acct = acct # Instance data — these trigger prop setters too!
        self.name = name # X mangled to have class name
        self.age = age
        self.addr = addr # addr is not managed; remain has no data
        def getName(self): return self.name
        def setName(self, value):
            value = value.lower().replace(" ", "")
            self.name = value
            name = property(getName, setName) # Or @ decorators for both
        def getAge(self): return self.age
        def setAge(self, value):
            if value < 0 or value > 150: raise ValueError('invalid age')
            else: self.age = value
            age = property(getAge, setAge)
        def getAcct(self): return self.acct[-3]
        def setAcct(self, value):
            value = value.replace('-', '')
            if len(value) != self.acctlen:
                raise TypeError('invalid acct number')
            else: self.acct = value
            acct = property(getAcct, setAcct)
        def remainGet(self): # Could be a method, not attr
            return self.retireage - self.age # Unless already using as attr
            remain = property(
```

remainGet)

中文翻译

示例 38-19 首先用 property 管理三个属性。当然也可用普通方法，但若既有代码已在用属性，property 更有帮助。property 在访问时自动运行代码，但聚焦特定属性集，不能泛化拦截所有属性。理解代码的关键：init 内的属性赋值也会触发 property 的 setter。例如 self.name = ... 自动调用 setName，变换值后赋给实例属性 name，以免与 property 名冲突。这种重命名（有时称 name mangling）很重要，因为 property 共用实例状态、自身无状态。数据在 name，而 name 永远是 property 而非数据。如第 31 章，name 这类名是伪私有（pseudoprivate）属性，存入实例命名空间时会被 Python 改写成含所属类名；这里有助于把实现专用属性与其他名字（含管理它们的 property）区分开。最终：管理 name、age、acct；addr 直接访问；只读虚属性 remain 按需计算。对比起见，property 版约 39 行（含空行）。

深度理解

- 核心概念：init 走 setter = 构造期也应用同一校验/变换规则（单一路径）。
- 底层实现：name → CardHoldername，避免与 property name 及子类撞名。
- 设计原因：对外 API 稳定为属性访问；对内存存储换名。
- 实际问题：账号显示掩码、姓名规范化、年龄校验、退休剩余年数。

·初学者误区：在 setter 里再写 `self.name = ...` 而非 `self.name`——无限递归。

代码分析

```
def setName(self, value):  
    value = value.lower().replace(' ', '_')  
    self.__name = value    # self.name  
name = property(getName, setName)
```

· remain 无 setter → 只读；赋值会 `AttributeError`。

38.35 Testing code (测试代码)

英文原文

To test our class, run the script in Example 38-20 in a console with the name of the class's module (sans `".py"`) as a single command-line argument. We'll use this same test script for all four versions of this example so their output will be the same. When it runs, it makes two instances of our managed-attribute class and fetches and changes their various attributes. Operations expected to fail are wrapped in try statements.

Example 38-20. `validatetester.py`

```
python def loadclass(): import sys, importlib  
module = sys.argv[1] # Module name in command line  
module = importlib.importmodule(modulename) # I  
import module by name string print(f'[Using: {module  
.CardHolder}])' # No need for getattr() here return m
```

```
odule.CardHolder def printholder(who): print(who.ac  
ct, who.name, who.age, who.remain, who.addr, sep='  
/') if name == 'main': CardHolder = loadclass() bob =  
CardHolder('1234-5678','Bob Smith', 40,'123 main st')  
printholder(bob)
```

```
bob.name = 'Bob Q. Smith' bob.age = 50 bob.acct = '2  
3-45-67-89' printholder(bob)
```

```
sue = CardHolder('5678-12-34','Sue Jones', 35,'124 m  
ain st') printholder(sue)
```

```
try: sue.age = 200 except: print('Bad age for Sue') try:  
sue.remain = 5 except: print("Can't set sue.remain") t  
ry: sue.acct = '1234567' except: print('Bad acct for Sue'  
) Following is the output of our test script's code; aga
```

in, this is the same for the other versions of this exam
ple ahead, except for the tested class's name. Accoun
ts are displayed with some digits hidden, names are c
onverted to a standard format, and time remaining u
ntil retirement (hypothetically speaking) is computed
when fetched using a class-attribute cutoff:

```
$ python3 validate tester.py validate properties [Usin  
g: <class 'validateproperties.CardHolder'>] 12345 / b  
obsmith / 40 / 22.5 / 123 main st 23456 / bobq.smith  
/ 50 / 12.5 / 123 main st 56781 / suejones / 35 / 27.5  
/ 124 main st Bad age for Sue Can't set sue.remain Ba  
d acct for Sue
```

中文翻译

测试类时，在控制台运行示例 38-20，并把类模块名（不含 .py）作为唯一命令行参数。四个版本共用此测试脚本，输出相同。运行时创建两个受管属性类实例，读写其各属性；预期失败的操作包在 try 中。输出（除类名外各版相同）：账号部分数字隐藏，姓名转为标准格式，距退休剩余时间（假设）在读取时用类属性截止值计算。

深度理解

- 核心概念：importlib.import_module 按字符串动态加载——便于四模块对照。
- 底层实现：同一测试驱动不同实现，隔离“业务结果”与“实现机制”。
- 设计原因：回归时改一处测试，四实现一起验。
- 实际问题：插件式校验后端。
- 初学者误区：四个文件复制粘贴测试代码——日后改测试要改四处。

代码分析

```
module = importlib.import_module(sys.argv[1])  
return module.CardHolder
```

- 命令行：python3 validatetester.py validateproperties。

38.36 Using Descriptors to Validate (用描述符做校验)

英文原文

Now, let's recode our example using descriptors instead of properties. As we've seen, descriptors are very similar to properties in terms of functionality and roles; in fact, properties are basically a focused form of descriptor. Like properties, descriptors are designed to handle specific attributes, not generic attribute access. Unlike properties, descriptors can also have their own state, and so are perhaps a more general scheme.

中文翻译

现在用描述符重写示例。就功能与角色而言，描述符与 property 非常相似；事实上 property 基本上是聚焦版描述符。与 property 一样，描述符面向特定属性，而非通用访问。与 property 不同，描述符还可有自有状态，因而或许更通用的方案。

深度理解

- 核心概念：描述符版可把状态放在描述符或实例——选择决定正确性。
- 底层实现：嵌套类 Name/Age/Acct/Remain 作为 Card Holder 局部名。
- 设计原因：每个字段一个小类，结构清晰、可复用。

- 实际问题：字段库（EmailField、AgeField）。
 - 初学者误区：Option 1 共享描述符状态导致多实例串数据。
-

38.37 Option 1: shared descriptor-instance state (badly!) (选项 1：共享描述符状态——错误示范)

英文原文

To understand the code in Example 38-21, it's again important to notice that the attribute assignments inside the init constructor method trigger descriptor set methods. When the constructor method assigns to self.name, for example, it automatically invokes the Name.set() method, which transforms the value and assigns it to a descriptor attribute called name. In the end, this class implements the same attributes as the prior version: it manages name, age, and acct; allows a ddr directly; and provides read-only virtual remain. Notice how we must catch assignments to the remain name in its descriptor and raise an exception; as we learned earlier, if we did not do this, assigning to this attribute of an instance would silently create an instance attribute that hides the class-attribute descriptor. For comparison purposes, this descriptor-based coding takes 45 lines of code. Example 38-21. validatedes

criptors1.py

```
python class CardHolder: # Using shared descriptor s
tate acctlen = 8 retireage = 62.5 def init(self, acct, na
me, age, addr): self.acct = acct # These trigger set cal
ls too! self.name = name # X not needed: in descript
or self.age = age self.addr = addr class Name: def ge
t(self, instance, owner): return self.name # Class nam
es: CardHolder locals def set(self, instance, value): val
ue = value.lower().replace(",") self.name = value nam
e = Name() class Age: def get(self, instance, owner): r
eturn self.age # Use descriptor data def set(self, insta
nce, value): if value < 0 or value > 150: raise ValueErr
or('invalid age') else: self.age = value age = Age() cla
ss Acct: def get(self, instance, owner): return self.acct[
:-3] + "def set(self, instance, value): value = value.repl
ace('-',") if len(value) != instance.acctlen: # Use instan
ce class data raise TypeError('invalid acct number') el
se: self.acct = value acct = Acct() class Remain: def ge
t(self, instance, owner): return instance.retireage - ins
tance.age # Triggers Age.get def set(self, instance, va
lue): raise TypeError('cannot set remain') # Else set all
owed here remain = Remain() When run with the prio
r testing script, all examples in this section produce t
he same output as shown for properties earlier, exce
pt that the name of the class in the first line varies. O
ption 2 prelude — the shared-state bug: Unlike the p
roperty-based variant, in Example 38-21 the actual na
me value is attached to the descriptor object, not the
```

client class instance. Although we could store this value in either instance or descriptor state, the latter avoids the need to mangle names with underscores to avoid collisions. In the `CardHolder` client class, the attribute called `name` is always a descriptor object, not data. Importantly, the downside of this scheme is that state stored inside a descriptor itself is class-level data that is effectively shared by all client-class instances and so cannot vary between them. That is, storing state in the descriptor instance instead of the owner (client) class instance means that the state will be the same in all owner-class instances. Descriptor state can vary only per attribute appearance. To see this at work, try printing attributes of the `bob` instance after creating the second instance, `sue`, with the new test script in Example 38-22. The values of `sue`'s managed attributes (`name`, `age`, and `acct`) overwrite those of the earlier object `bob`, because both share the same, single descriptor instance attached to their class. Example 38-22. `validatetesterplus.py`

```
python from validatetester import loadclass
CardHolder = loadclass()
bob = CardHolder('1234-5678','Bob Smith', 40,'123 main st')
print('bob:', bob.name, bob.acct, bob.age, bob.addr)
sue = CardHolder('5678-12-34','Sue Jones', 35,'124 main st')
print('sue:', sue.name, sue.acct, sue.age, sue.addr)
# addr differs: client data
print('bob:', bob.name, bob.acct, bob.age, bob.addr)
# name,acct,age overwritten?
When this script is run
```

with the descriptor-state `CardHolder` of Example 38-21, the results confirm the suspicion—in terms of managed attributes, bob has morphed into sue!

```
$ python3 validatetesterplus.py validatedescriptors1
[Using: <class'validatedescriptors1.CardHolder'>] bo
b: bobsmith 12345 40 123 main st sue: suejones 5678
1 35 124 main st bob: suejones 56781 35 123 main st
```

This isn't an issue for properties because they have no state of their own, and there are valid uses for descriptor state. Such state might be used, for example, to manage descriptor implementation and data that spans all instances, and this example was coded this way on purpose to illustrate the technique. Moreover, the state scope implications of class versus instance attributes should be more or less a given at this point in the book. However, in this particular use case, attributes of `CardHolder` objects are probably better stored as per-instance data instead of descriptor-instance data, perhaps using the same X naming convention as the property-based equivalent to avoid name clashes in the instance—a more important factor this time, as the client is a different class with its own state attributes.

中文翻译

理解示例 38-21 同样关键：init 内赋值会触发描述符 set。例如 `self.name = ...` 自动调用 `Name.set()`，变换后把值赋给描述符属性 `name`。最终属性集合与 `property` 版相同。

注意必须在 Remain 的 set 中捕获赋值并抛异常；否则对实例该属性赋值会静默创建实例属性并遮蔽类上描述符。描述符版约 45 行。共享状态的坑：与 property 版不同，真实 name 值附在描述符对象上，而非客户端实例。虽两种状态都能存，后者可避免下划线改名防冲突。在 CardHolder 中，name 永远是描述符对象而非数据。重要缺点：存在描述符自身的状态是类级数据，被所有客户端类实例有效共享，不能在实例间变化。描述符状态只能随“属性出现点”变化。用示例 38-22 在创建 sue 后再打印 bob：会发现 bob 的受管属性被 sue 覆盖——二者共享类上同一个描述符实例。addr 仍不同（客户端实例数据）。property 无此问题（自身无状态）；描述符状态有合法用途（实现细节、跨实例数据），本例有意如此编写以演示。但本用例中，CardHolder 属性更宜作 per-instance 数据，或与 property 版相同的 X 约定避免实例名冲突——因客户端是另一带自有状态的类，这点更重要。

深度理解

- 核心概念：bob 变成 sue 是全书最该记住的描述符反例之一。
- 底层实现：CardHolder.name 只有一个 Name(); self.name = value 写的是描述符的 self.name。
- 设计原因：故意写错示范，对照 Option 2。
- 实际问题：多用户/多卡片对象时数据串台 = 严重 bug。
- 初学者误区：看到“不必 X 改名”就用共享状态——省事换来逻辑错误。

代码分析

```
class Name:
    def __set__(self, instance, value):
        self.name = value    # →
```

· addr 不受管，仍 per-instance，故最后一行 bob 的地址仍是自己的。

38.38 Option 2: per-client-instance state (correctly) (选项 2：每客户端实例状态——正确)

英文原文

Example 38-23 has the required changes; it doesn't change line counts (we're still at 45).

Example 38-23. validatedescriptors2.py

```
python class CardHolder: # Using per-client-instance
state acctlen = 8 retireage = 62.5 def init(self, acct, n
ame, age, addr): self.acct = acct # Client instance dat
a self.name = name # These trigger set calls too! self.
age = age # X needed: in client instance self.addr = a
ddr class Name: def get(self, instance, owner): return
instance.name def set(self, instance, value): value = v
alue.lower().replace(",") instance.name = value

name = Name() # class.name vs mangled attr class A
ge: def get(self, instance, owner): return instance.age
```

```
# Use instance data def set(self, instance, value): if va  
lue < 0 or value > 150: raise ValueError('invalid age')  
else: instance.age = value
```

```
age = Age() class Acct: def get(self, instance, owner):  
return instance.acct[:-3] + "def set(self, instance, valu  
e): value = value.replace('-', '') if len(value) != instance  
.acctlen: raise TypeError('invalid acct number') else: in  
stance.acct = value
```

```
acct = Acct() class Remain: def get(self, instance, own  
er): return instance.retireage - instance.age # Trigger  
s Age.get def set(self, instance, value): raise TypeErro  
r('cannot set remain') remain = Remain() This support  
s per-instance data for the managed fields as expecte  
d (bob remains bob), and other tests work as before:
```

```
$ python3 validatetesterplus.py validatedescriptors2  
bob: bobsmith 12345 40 123 main st sue: suejones 5  
6781 35 124 main st bob: bobsmith 12345 40 123 ma  
in st
```

One small caveat here: as coded, this version doesn't support through-class descriptor access because such access passes a None to the instance argument (name mangling appears in the error as `Namename`):

```
python >> CardHolder.name # after creating an insta  
nce with descriptors2 AttributeError:'NoneType' obje  
ct has no attribute'Namename'
```

We could detect this with a minor amount of additio

nal code to trigger the error more explicitly, but there's probably no point—because this version stores data in the client instance, there's no meaning to its descriptors unless they're accompanied by a client instance (much like a normal unbound instance method). In fact, that's really the entire point of this version's change!

Because they are classes, descriptors are a useful and powerful tool, but they present choices that can deeply impact a program's behavior. As always in OOP, choose your state retention policies carefully.

中文翻译

示例 38-23 做了必要修改，行数仍约 45。关键变化：get/set 读写 `instance.name` 等，而非 `self.xxx`。这按预期支持受管字段的 per-instance 数据（bob 仍是 bob），其他测试同前。小警告：此版不支持通过类访问描述符，因为此时 `instance` 为 `None`（错误信息中可见 name mangling 后的 `Namename`）。可加少量代码让错误更显式，但多半无必要——数据在客户端实例，没有实例时描述符无意义（很像未绑定的普通实例方法）。这正是本版改动的全部意义！描述符作为类很有用、很强大，但状态选择会深刻影响行为。与 OOP 一贯一样：谨慎选择状态保留策略。

深度理解

- 核心概念：正确版 = 行为在描述符，数据在实例。
- 底层实现：嵌套类中的 `name` 会 mangling 成 `Namena`

me (以嵌套类名为准)。

- 设计原因：与 property 的伪私有存储同构，但每个字段是独立类。
- 实际问题：多卡片、多用户场景的正确模型。
- 初学者误区：对 CardHolder.name 做类级访问期望得到“当前值”——应返回描述符或显式拒绝。

代码分析

```
def __set__(self, instance, value):  
    instance.__name = value #
```

- validate 下 bob 不再被 sue 覆盖。

38.39 Using getattr to Validate (用 getattr 做校验)

英文原文

As we've seen, the getattr method intercepts all unde-fined attributes, so it can be more generic than using properties or descriptors. For our example, we simply test the attribute name to know when a managed attribute is being fetched; others are stored physically on the instance and so never reach getattr.

Although this approach is more general than using properties or descriptors, extra work may be required to imitate the specific attribute focus of other tools. We need to check names at runtime—a multiple-choi

ce that's a prime role for the match statement—and we must code a `setattr` in order to intercept and validate attribute assignments.

Example 38-24 hosts the `getattr` version of our validations code. In the end, this class, like the prior two, manages `name`, `age`, and `acct`; allows `addr` directly; and provides read-only virtual `remain`.

As for the property and descriptor versions, it's critical to notice that the attribute assignments inside the `init` constructor method trigger the class's `setattr` method too. When this method assigns to `self.name`, for example, it automatically invokes `setattr`, which transforms the value and assigns it to an instance attribute called `name`.

By storing `name` on the instance, it ensures that future accesses will not trigger `getattr`. In contrast, `acct` is stored as `acct` so that later accesses to `acct` do invoke `getattr`.

For comparison purposes, this alternative comes in at 34 lines of code—5 fewer than the property-based version and 11 fewer than the version using descriptors (though replacing `if` with `match` here added two lines, along with extra indentation). Clarity matters more than code size, of course, but extra code can imply extra development and maintenance work.

Probably more important here are roles: generic tools like `getattr` are better suited to generic delegation,

while properties and descriptors are designed to manage specific attributes.

Also note again that the code here incurs extra calls when setting unmanaged attributes (e.g., `addr`), although no extra calls are incurred for fetching unmanaged attributes since they are defined. Though this will likely result in negligible overhead for most programs, the more narrowly focused properties and descriptors incur an extra call only when managed attributes are accessed, and also appear in `dir` results automatically when needed by generic tools.

Example 38-24. `validategetattr.py`

```
python class CardHolder: acctlen = 8 retireage = 62.5
def init(self, acct, name, age, addr): self.acct = acct #
These trigger setattr too self.name = name # acct not
mangled: name tested self.age = age self.addr = addr # addr is not managed; remain has no data
def getattr(self, name): match name: # On undefined attr fetches case 'acct':
return self.acct[-3] + ' ' # name, age, addr are defined
case 'remain':
return self.retireage - self.age # Doesn't trigger setattr case :
raise AttributeError(name) def setattr(self, name, value): match name: # On all attr assignments case 'name':
:
```

```
value = value.lower().replace('"','') # addr stored directly
case'age':

if value < 0 or value > 150: raise ValueError('invalid age')
case'acct':

name = 'acct' # acct mangled to acct
value = value.replace('-', '')
if len(value) != self.acctlen: raise TypeError('invalid acct number')
case'remain':

raise TypeError('cannot set remain')
self.dict[name] = value # Avoid looping (or object)
When this code is run with either test script, it produces the same output (with a different class name).
```

中文翻译

如前所述，`getattr` 拦截所有未定义属性，因而可比 `property`/描述符更通用。本例只需测试属性名以判断何时在读取受管属性；其他属性物理存在于实例上，永不进入 `getattr`。虽更通用，但要模仿其他工具的“特定属性焦点”可能需要额外工作：运行时检查名字（`match` 的理想用途），且必须编写 `setattr` 拦截并校验赋值。示例 38-24 是 `getattr` 版校验。属性集合同前。关键点：`init` 赋值也会触发 `setattr`。例如 `self.name = ...` 进入 `setattr`，变换后把值存为实例属性 `name`——从而保证之后读取 `name` 不会再进 `getattr`。相反，`acct` 存为 `acct`，使之后读 `acct` 会进 `getattr`（以便掩码）。对比：约 34 行——比 `property` 少 5、比描述符少 11（`match` 多 2 行及缩进）。清晰比行数更重要，但多余代码可能意味多余开发与维护。更重要的是角色：`getattr` 更适通用委托；`property`/描述符适管理特定属性。

另：对未管理属性的赋值会产生额外调用（如 `addr`）；读取未管理属性因已定义而无额外调用。多数程序开销可忽略；更聚焦的 `property`/描述符仅在访问受管属性时额外调用，且自动出现在 `dir` 中。

深度理解

- 核心概念：存不存实例上 = 是否进 `getattr`——用存储策略控制拦截。
- 底层实现：`name/age` 存真值（读不拦截）；`acct` 存 `ac` `ct`（读时掩码）；`remain` 完全不存（读必拦截）。
- 设计原因：读路径按需拦截，写路径统一校验。
- 实际问题：行数更少，但 `dir` 看不到虚名、未管理写有额外成本。
- 初学者误区：所有字段都存成带下划线再全部走 `getattr`——失去“未管理读零成本”。

代码分析

```
case 'acct':  
    name = '_acct'  
    value = value.replace('-', '')  
    ...  
self.__dict__[name] = value
```

- 写 `acct` 时改名存 `acct`；读 `acct` 未定义 → `getattr` → 掩码返回。
-

38.40 Using `getattribute` to Validate (用 `getattribute` 做校验)

英文原文

Our final variant uses the `getattribute` catchall to intercept attribute fetches and manage them as needed. Every attribute fetch is caught here, so we test the attribute names to detect managed attributes and route all others to the superclass for normal fetch processing. This version uses the same `setattr` to catch assignments as the prior version (there is no corresponding “`setattribute`” in Python—so far?).

Example 38-25 codes this last mod. It works very much like the `getattr` version, so we won't repeat the full description here. Note, though, that because every attribute fetch is routed to `getattribute`, we don't need to mangle names to intercept them here (`acct` is stored as `acct`). On the other hand, this code must take care to route nonmanaged attribute fetches to a superclass to avoid looping or extra calls.

Also, notice that this version incurs extra calls for both setting and fetching unmanaged attributes (e.g., `addr`); if speed is paramount, this alternative may be the slowest of the bunch. For comparison purposes, this version amounts to 34 lines of code, just like the prior version (and again including 2 lines added by match

).

Example 38-25. validategetattribute.py

```
python class CardHolder: acctlen = 8 retireage = 62.5
def init(self, acct, name, age, addr): self.acct = acct #
These trigger setattr too self.name = name # acct not
mangled: name tested self.age = age self.addr = add
r def getattribute(self, name): superget = object.getat
tribute # Don't loop: level up match name: # On all at
tr fetches case'acct':

return superget(self,'acct')[:-3] + "case'remain':

return superget(self,'retireage') - superget(self,'age')
case :

return superget(self, name) # name, age, addr: stored
def setattr(self, name, value): match name: # On all at
tr assignments case'name':

value = value.lower().replace(",") case'age':

if value < 0 or value > 150: raise ValueError('invalid a
ge') case'acct':

value = value.replace('-',") if len(value) != self.acctlen:
raise TypeError('invalid acct number') case'remain':

raise TypeError('cannot set remain') self.dict[name] =
value # Avoid loop, orig names Both the setattr and
getattribute scripts work the same as the property an
d per-client-instance descriptor versions when run by
both tester scripts—four ways to achieve the same g
```

oal in Python, though they vary in structure and are perhaps less redundant in some other roles. Be sure to study and run this section's code on your own for more pointers on managed-attribute coding techniques.

中文翻译

最终变体用 `getattr` 兜底拦截属性读取并按需管理。这里捕获每一次读取，故测试属性名以识别受管属性，并把其余交给超类做正常读取。赋值仍用与上一版相同的 `setattr` (Python 尚无对应的 “`setattribute`” ——目前?)。示例 38-25 编码最后一版。工作方式与 `getattr` 版非常像。注意：因每次读取都进 `getattr`，此处不必为拦截而改名 (acct 就存为 acct)。另一方面，必须把未管理读取路由到超类，以免循环或额外调用。另：此版对未管理属性的读与写都会产生额外调用 (如 `addr`)；若速度至上，可能是最慢的。行数同样约 34。`getattr` 与 `getattr` 脚本在两种测试下与 `property`、`per-instance` 描述符版结果相同——Python 中达成同一目标的四种方式，结构各异，在其他角色中冗余或许更少。请务必自行研读并运行本节代码。

深度理解

- 核心概念：全拦截 → 不必 acct 技巧，但未管理读写都有税。
- 底层实现：`superget = object.getattribute` 局部绑定，避免循环且少写长名。
- 设计原因：统一入口做审计/权限时很香；做 `CardHolder`

r 校验则可能过重。

·实际问题：四方式对照表是选型决策树。

·初学者误区：setattr 里 len(value) != self.acctlen 会触发 getattrattribute('acctlen')——此处类属性，一般无害，但是否循环取决于实现。

代码分析

```
def __getattrattribute__(self, name):
    superget = object.__getattrattribute__
    match name:
        case 'acct':
            return superget(self, 'acct')[:-3] + '***'
        case 'remain':
            return superget(self, 'retireage') - superget(self, 'a...
        case _:
            return superget(self, name)
```

·remain 通过 superget 取 retireage 与 age，避免 self.age 再入时的额外逻辑纠缠（age 存真值，走 default 分支亦可，但 superget 更清晰）。

38.41 Chapter Summary (本章小结)

英文原文

This chapter covered the various techniques for managing access to attributes in Python, including the setattr and getattrattribute operator-overloading methods, and class properties and descriptors. Along the way, it compared and contrasted these tools and presented

d a handful of use cases to demonstrate their behavior. Chapter 39 continues our tool-building focus with a survey of decorators—code run automatically at function and class creation time rather than on attribute access. Before we continue, though, let's work through a set of questions to review what we've covered here.

中文翻译

本章涵盖了 Python 中管理属性访问的多种技术，包括运算符重载方法 `getattr` 与 `getattrattribute`，以及类的 `property` 与描述符。过程中对比了这些工具，并给出若干用例演示其行为。第 39 章将继续工具构建焦点，纵览装饰器——在函数与类创建时而非属性访问时自动运行的代码。继续之前，先用一组问题复习本章内容。

深度理解

- 核心概念：四工具目标重叠、触发条件与范围不同。
- 底层实现：描述符协议 vs 运算符重载，两条实现路线。
- 设计原因：为“单字段”与“全代理”提供各自优雅路径。
- 实际问题：下一章装饰器会复用描述符与属性拦截。
- 初学者误区：把本章当“必须全会写”——应用开发者读懂即可，工具作者才需全写。

38.42 Test Your Knowledge: Quiz (知识检测：测验)

英文原文

1. How do getattr and getattribute differ? 2. How do properties and descriptors differ? 3. How are properties and decorators related? 4. What are the main functional differences between getattr and getattribute and properties and descriptors?

中文翻译

1. getattr 与 getattribute 有何不同？ 2. property 与描述符有何不同？ 3. property 与装饰器如何相关？ 4. getattr/getattribute 与 property/描述符的主要功能差异是什么？

深度理解

·自测焦点：触发条件、状态、通用性、赋值是否一并处理、dir 可见性、内建运算。

38.43 Test Your Knowledge: Answers (知识检测：答案)

英文原文

1. The getattr method is run for explicit fetches of undefined attributes only (i.e., those not present on an i

nstance and not inherited from any of its classes). By contrast, the `getattr` method is called for every explicit attribute fetch, whether the attribute is defined or not. Because of this, code inside a `getattr` can freely fetch other attributes if they are defined, whereas `getAttribute` must use special code for all such attribute fetches to avoid looping or extra calls (it must route fetches to a superclass to skip itself). Neither method is run for implicit fetches of built-in operations (sans the next chapter's heroics).

2. Properties serve a specific role, whereas descriptors are more general. Properties define `get`, `set`, and `delete` functions for a specific attribute; descriptors provide a class with methods for these actions, too, but they provide extra flexibility to support more arbitrary actions. In fact, properties are really a simple way to create a specific kind of descriptor—one that runs functions on attribute accesses. Coding differs too: a property is created with a built-in function, and a descriptor is coded with a class; thus, descriptors can leverage all the usual OOP features of classes, such as inheritance. Moreover, in addition to the instance's state information, descriptors have local state of their own, which can sometimes avoid name collisions in the instance.

3. Properties can be coded with decorator syntax. Because the property built-in accepts a single function argument and returns a function, it can be used directly as a function decorator to define a fetch-access property. Due to t

he name rebinding behavior of decorators, the name of the decorated function is assigned to a property whose get accessor is set to the original function decorated (`name = property(name)`). Property setter and deleter attributes allow us to further add set and delete accessors with decoration syntax—they set the accessor to the decorated function and return the augmented property. Some may find this a bit clumsy, but this is subjective. 4. The `getattr` and `getattribute` methods are more generic: they can be used to catch arbitrarily many attributes. In contrast, each property or descriptor provides access interception for only one specific attribute—we can't catch every attribute fetch with a single property or descriptor. On the other hand, properties and descriptors handle both attribute fetch and assignment by design: `getattr` and `getattribute` handle fetches only; to intercept assignments as well, `setattr` must also be coded. The implementation is also different: `getattr` and `getattribute` are operator-overloading methods, whereas properties and descriptors are objects manually assigned to class attributes. Unlike the others, properties and descriptors can also sometimes avoid extra calls on assignment to unmanaged names and show up in `dir` results automatically, but are also narrower in scope—they can't address generic delegation goals. In Python evolution, new features tend to offer alternatives but often do not fully subsume what came before.

中文翻译

1. `getattr` 仅在显式读取未定义属性时运行（实例上没有、也未从任何类继承）。`getattribute` 则对每一次显式属性读取调用，无论是否已定义。因此 `getattr` 内可自由读取其他已定义属性；`getattribute` 对所有此类读取必须用特殊代码避免循环或额外调用（须路由到超类以跳过自身）。二者都不会因内建运算的隐式读取而运行（除非下一章的“英雄式”变通）。2. `property` 角色更特定，描述符更通用。`property` 为特定属性定义 `get/set/delete` 函数；描述符也提供含这些动作方法的类，但更灵活以支持更任意的行为。事实上 `property` 只是创建特定种类描述符（在属性访问时运行函数）的简单方式。编码也不同：`property` 用内建函数创建，描述符用类编写，因而描述符可利用类的全部 OOP 特性（如继承）。此外，除实例状态外，描述符还有本地状态，有时可避免实例中的名字冲突。3. `property` 可用装饰器语法编码。因 `property` 内建接受单一函数参数并返回可调对象，可直接作函数装饰器定义读取访问 `property`。因装饰器的名字重绑定，被装饰函数名被赋给一个 `get` 访问器为原函数的 `property` (`name = property(name)`)。`property` 的 `setter/deleter` 属性允许继续用装饰语法添加 `set/delete` 访问器——它们把访问器设为被装饰函数并返回增强后的 `property`。有人觉得略笨拙，但这是主观的。4. `getattr / getattribute` 更通用：可捕获任意多个属性。相比之下，每个 `property` 或描述符只拦截一个特定属性——不能用单个 `property`/描述符捕获每一次属性读取。另一方面，`property` 与描述符在设计上同时处理读取与赋值；`getattr / getattribute` 只处理读取，要拦截赋值还须编写 `set`

attr。实现也不同：前二者是运算符重载方法，后二者是手动赋给类属性的对象。与其他不同，property/描述符有时还能避免对未管理名赋值的额外调用，并自动出现在 dir 结果中，但作用域也更窄——无法解决通用委托目标。在 Python 演化中，新特性往往提供替代方案，但常常并不完全取代旧方案。

深度理解

- 核心概念：答案四条几乎是本章的“压缩饼干”——背熟选型就不慌。
- 底层实现：记得“数据描述符 > 实例 dict > 非数据描述符 > getattr”。
- 设计原因：新工具叠加而非替换——Python 一贯风格。
- 实际问题：面试/读源码时这四问是高频考点。
- 初学者误区：以为有了描述符就可以忘掉 getattr——委托场景仍然要它。

本章总结

技术拓展 (Technical Expansion)

- 实际项目中的应用场景
- Django/SQLAlchemy 模型字段：描述符（或类描述符）在类上定义列，实例上存行数据。
- @property 做只读视图、延迟计算（fullname、area）、带校验的配置项。

- 远程/RPC 代理、测试 Mock: getattr 把任意方法名转发给真实对象。
- 访问审计、权限: getattribute/setattr 记录或拒绝敏感字段。
- 敏感信息掩码 (账号、手机号): 读时变换, 与 CardHolder 的 acct 同构。
- 与其他语言 (Java/C++) 的区别

语言	类似机制	本质	与 Python 差异
Python	property / descriptor / getattr	运行时协议	语法仍是 obj.x, 钩子可插拔
Java	getter/setter 方法	显式方法调用	调用方必须写 getX(), 改字段要改调用点
C#	property	编译为方法对	语法接近 Python property, 但是静态类型
C++	无统一属性协议	手工 operator 或方法	无描述符式“类属性劫持”标准机制

- Python 发展历史背景
- 经典类时代即有 getattr/setattr; 新式类完善 getattribute 与描述符协议。
- property 内建基于描述符; @property 装饰器语法随函数装饰器 (2.4+) 普及。
- 3.X 明确: 内建运算不走实例 getattribute, 形成委托代理的长期议题。
- slots、classmethod、staticmethod、绑定方法均建立在描述符协议之上。

·高级开发者需要掌握的相关知识

主题	作用
数据 vs 非数据描述符	决定与实例 dict 的优先级
object.getattribute / setattr	全拦截时的防递归标准手法
slots 与无 dict	迫使使用超类 setattr
functools.cachedproperty	现代缓存计算属性
第 39 章装饰器 + 描述符	方法装饰器、内建运算补丁
第 40 章元类	类创建时自动安装描述符

学习建议 (Learning Advice)

·重要程度：5/5 星（高级必读）。不懂描述符就读不懂半个 Python 框架；不懂 getattr 就写不好代理。

·应该掌握到什么程度

1. 能手写 property 与等价描述符，并说明差异。
2. 能解释为何“省略 set ≠ 只读”，并写出只读描述符。
3. 能在 getattribute/setattr 中正确防递归。
4. 能说明 3.X 内建运算为何不进 getattr，并知道补救方向（第 39 章）。
5. 能把 CardHolder 校验用至少两种技术实现，并指出描述符共享状态的错误。

·后续应该学习哪些相关内容

1. 第 39 章 Decorators——属性隐私、参数校验、内建运算委托修复。
2. 第 40 章 Metaclasses——类创建时安装描述符、完整继承与数据描述符优先级。

3. 重读第 30–32 章运算符重载、委托、slots、property 预告。
4. 阅读 CPython 中 property、function.get、slot 描述符相关实现或文档。
5. 动手：把 CardHolder 四实现跑通，并用 validate_test_erplus 复现 bob→sue bug。

全章完。管理属性的本质是：让 obj.x 的背后可以跑你的代码。property 与描述符精确到字段；getattr 与 getattribute 宽到代理。状态放错地方（描述符共享 vs 实例私有）会让对象“互相变形”——CardHolder 的教训请刻在脑子里。下一站：装饰器，把“定义结束时自动跑代码”从属性扩展到函数与类。